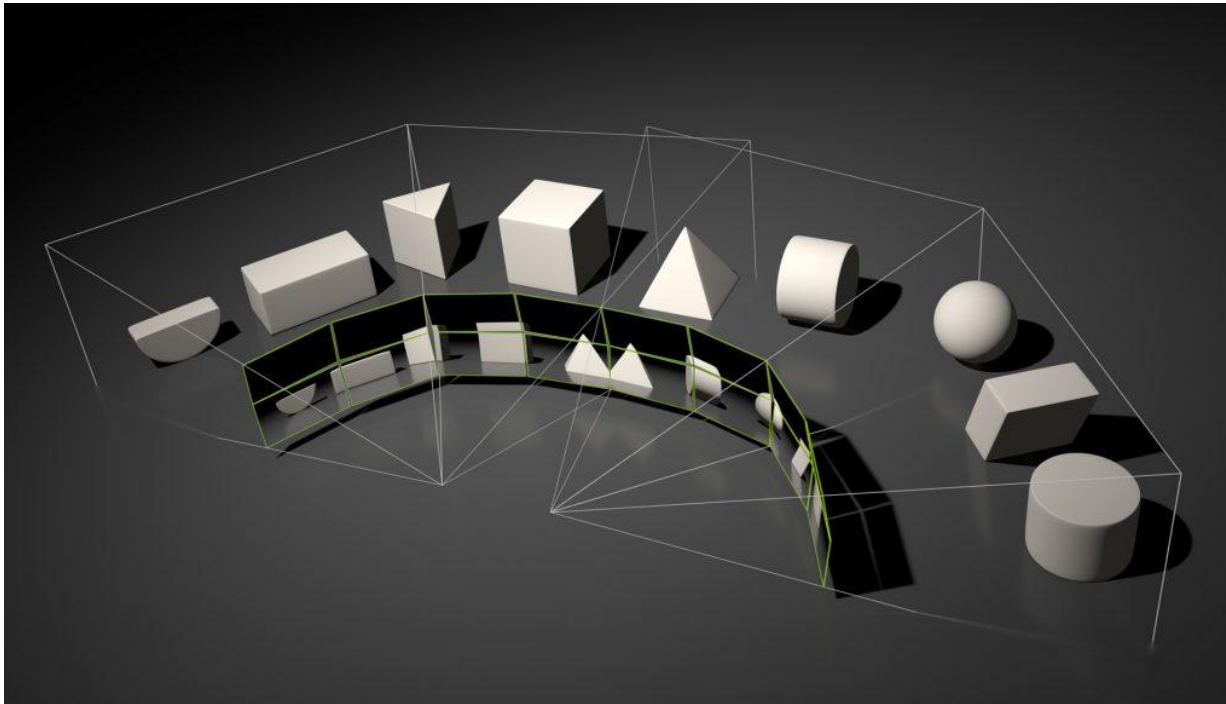


VR渲染



计算机系 张松海

题图：多视图渲染——NVIDIA GeForce RTX 显卡支持的VR渲染新特性 SIGGRAPH 2018

<https://www.roadtovr.com/nvidias-geforce-rtx-cards-bring-new-vr-rendering-features-and-enhancements/>

目录

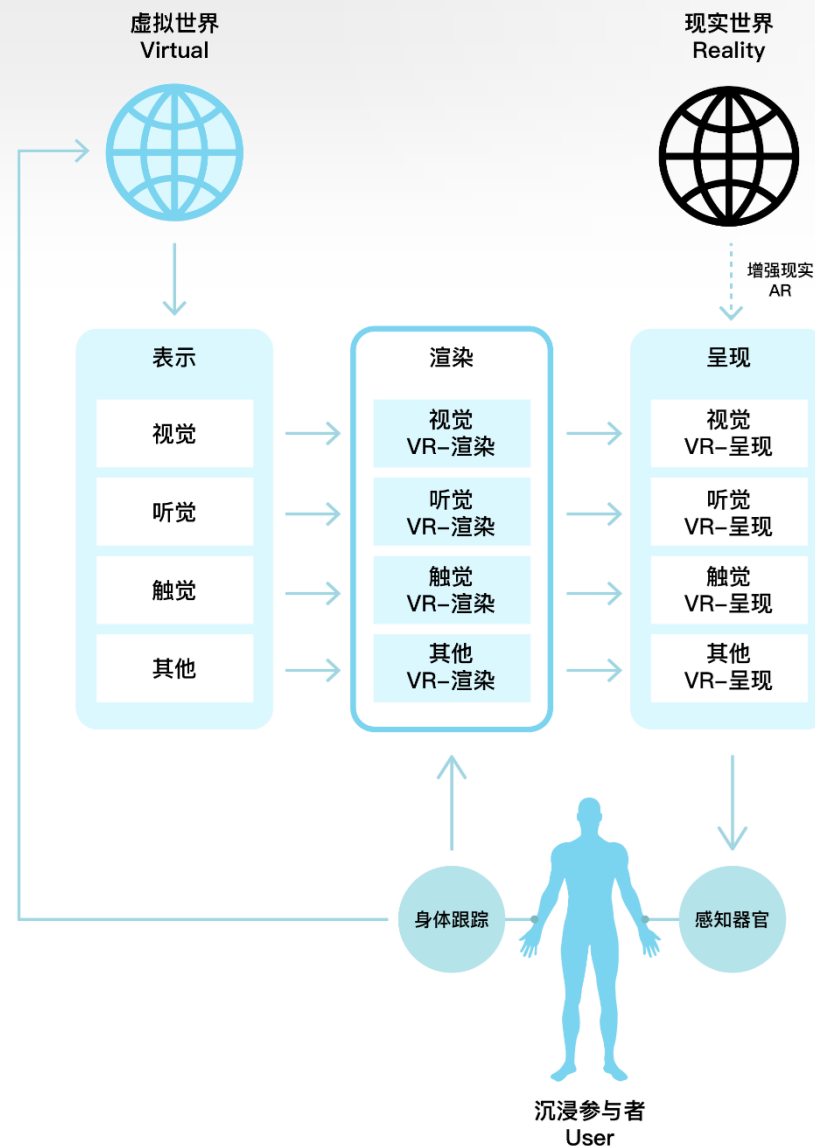
- 01 VR渲染引言
- 02 视觉渲染基本方法
- 03 针对VR的视觉渲染增强
- 04 VR中的其他感官渲染

目录

- 01 VR渲染引言
- 02 视觉渲染基本方法
- 03 针对VR的视觉渲染增强
- 04 VR中的其他感官渲染

什么是VR渲染 (Rendering)

- 完整的虚拟现实系统中, 每一种感知都要呈现给虚拟世界的参与者
- 渲染** (rendering) 是创建描述虚拟世界的感官信号的过程
- 这些感官信号可以是视觉、听觉、触觉或其他感官



VR对渲染的要求

- **实时性**

- 感官信号的渲染速度必须满足实时性，让用户感觉是连续的而不是离散的。
- 必须满足每一种感知模态的实时渲染要求，以及每一种的最小延迟阈值：视觉信号一般60-90Hz，听觉信号根据音质一般从8kHz到96kHz不等
- 实时的两个因素：延迟、效率

VR对渲染的要求

- **保真性**

- 高分辨率和真实感、大的视野、保真的声音

- **交互性**

- 渲染速度必须能够响应用户在VR中的移动和动作

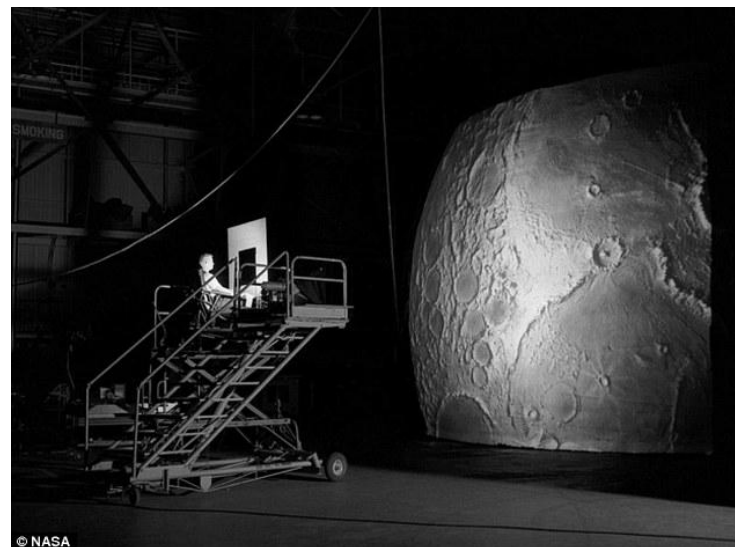
渲染硬件——从模拟到计算

- 通过模拟生成VR感官信号

- 在计算机图形学诞生之前，早期计算机**不足以**支撑VR系统所要求的逼真渲染，感官信号主要由模拟来生成。比如著名的阿波罗月球轨道着陆模拟器的视觉显示就是由预先录制的图像来模拟



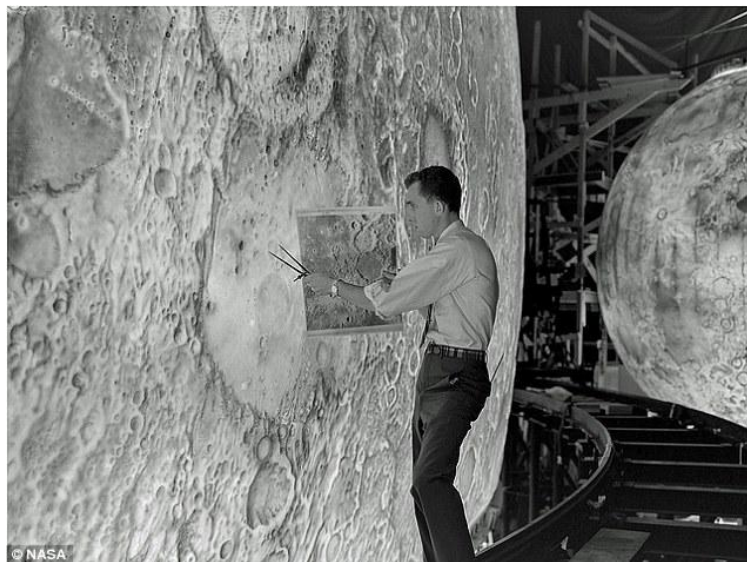
NASA月球轨道着陆模拟器(LOLA)



LOLA的视觉渲染由巨大图片来模拟

渲染硬件——从模拟到计算

- 通过模拟生成VR感官信号



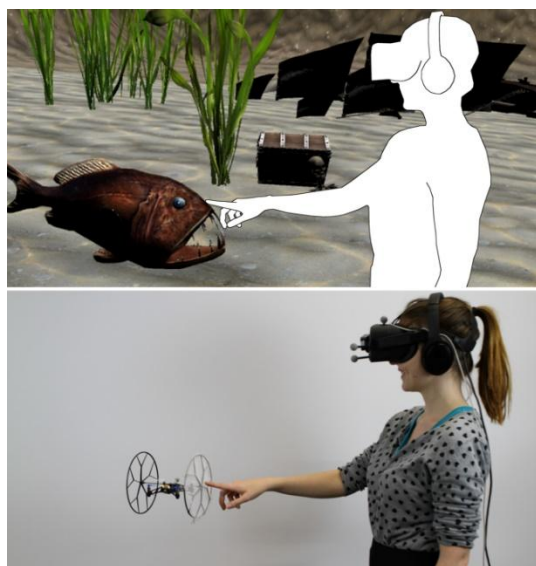
工作人员维护LOLA用于视觉显示的图像

<https://www.dailymail.co.uk/news/article-2237819/NASAs-ingenious-moon-simulator-helped-prepare-Apollo-astronauts-land-gone-before.html>

渲染硬件——从模拟到计算

- 通过模拟生成VR感官信号

- 在现代虚拟现实系统中，也并非所有的渲染都是计算机生成的虚拟信号



使用无人机模拟VR触觉信号

Hoppe, M., Knierim, P., Kosch, T., Funk, M., Futami, L., Schneegass, S., ... & Machulla, T. (2018, November). VRHapticDrones: Providing haptics in virtual reality through quadcopters. In *Proceedings of the 17th International Conference on Mobile and Ubiquitous Multimedia* (pp. 7-18).



4D影院中“风”的信号由真实的喷气来模拟

澳大利亚最大的 4D 电影院即将进驻悉尼
<https://www.flicks.com.au/news/australias-largest-4d-cinema-is-coming-to-sydney/>

渲染硬件——从模拟到计算

- **通过计算机生成VR感官信号**

- 在大型机时代，一些公司如 Evans & Sutherland 致力于视觉和声音渲染方面，并取得许多进展。
- 工作站时代，经济进一步推动了视觉计算的发展，硅谷图形公司（Silicon Graphics, Inc.）等的图形引擎实现了实时视觉渲染。
- 在声学领域，数字信号处理器芯片（DSP）可以用来实时处理声音信号。



Evans & Sutherland 飞行模拟 1972

<http://computergraphmuseum.free.fr/simulation.htm>



SGI™ 1400家庭工作站图形引擎 1984

<https://www.techspot.com/article/2142-silicon-graphics/>

渲染硬件——从模拟到计算

- 通过计算机生成VR感官信号
 - 台式机和移动计算机时代，图形引擎变成了单芯片图形处理单元（GPU）
 - 可编程图形单元在 2005 年左右开始出现在消费类 GPU 上
 - GPU 允许内部编程，不仅适用于视觉渲染，还可作为并行处理器，用于触觉，甚至碰撞检测和物理模拟

时间	80 年代	80 年代末	90 年代初	90 年代后期	2004 ~ 2010	2011 ~ 至今
类型	图形显示	2D 加速	部分 3D 加速	固定管线	统一渲染	通用计算
相关标准	CGA,VGA	GDI,DirectFB	OpenGL(1.1 ~ 4.1), DirectX(6.0 ~ 11)			CUDA,OpenCL1.2 ~ 2.0
代表产品	IBM 5150	86C911	Glnt300SX	GeForce256	G80	TESLA
基本特征	光栅生成器	2D 图元加速	硬件 T&L	shader 功能固定	多功能 shader	完成与图形处理无关的科学计算

GPU的发展历程

熊庭刚. GPU的发展历程、未来趋势及研制实践[J].微纳电子与智能制造, 2020, 2(2): 36-40.

VR渲染在哪里计算

- 外部计算机
- 头显内部
- 云渲染与远端渲染



Hodgson, E., Bachmann, E., & Thrash, T. (2014). Performance of redirected walking algorithms in a constrained virtual world. *IEEE transactions on visualization and computer graphics*, 20(4), 579-587.

VR渲染在哪里计算

• 外部计算机

- 通过连接本地计算机或佩戴背包式计算机作为渲染计算硬件
- 渲染与显示分离，通过线缆将渲染硬件和用户显示硬件连接
- 眼镜：动作感知器和显示屏；
计算机：内容生成器



用户佩戴背包式计算机进行VR漫游

Bachmann, E. R., Hodgson, E., Hoffbauer, C., & Messinger, J. (2019). Multi-user redirected walking and resetting using artificial potential fields. *IEEE transactions on visualization and computer graphics*, 25(5), 2022-2031.



用户通过线缆连接本地计算机进行VR漫游

<https://vrscout.com/news/cant-stop-playing-pool-nation-vr/>

VR渲染在哪里计算

• 头显内部

- 摆脱“小辫子”一直是VR的追求，将渲染计算迁移至眼镜内部
- 典型的例子：用智能手机作为VR显示和渲染设备，微软HoloLens等VR/AR一体机
- 算力不足，渲染效果受限



手机式VR 与 AR 移动平台

<https://shop.mergeedu.com/products/merge-ar-vr-headset?variant=32139015618627>



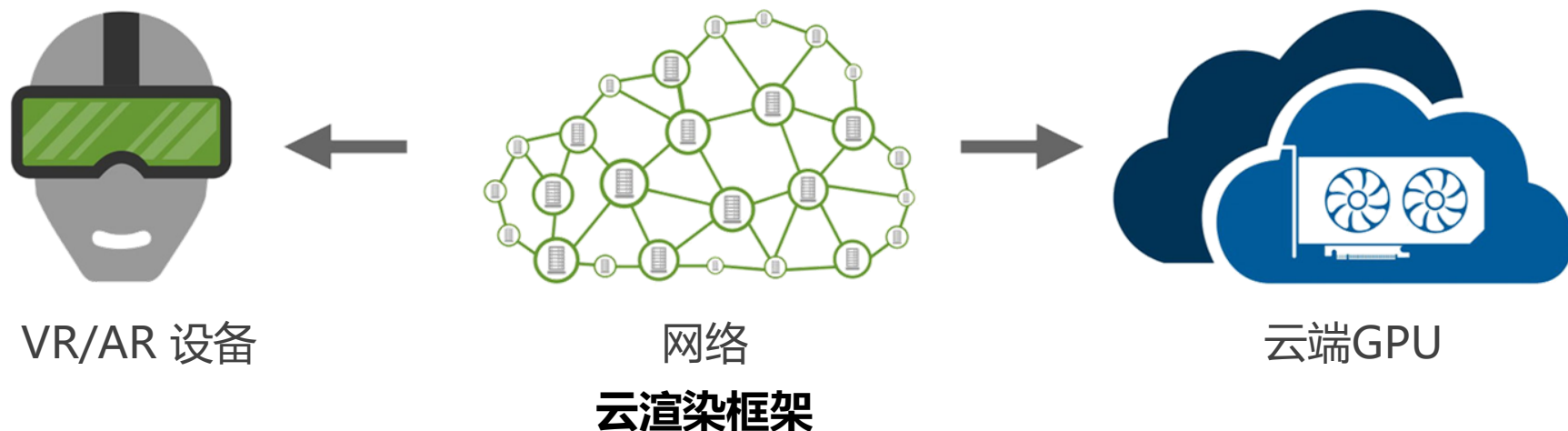
一体式VR 与 AR 移动平台

<https://www.microsoft.com/en-us/hololens>

VR渲染在哪里计算

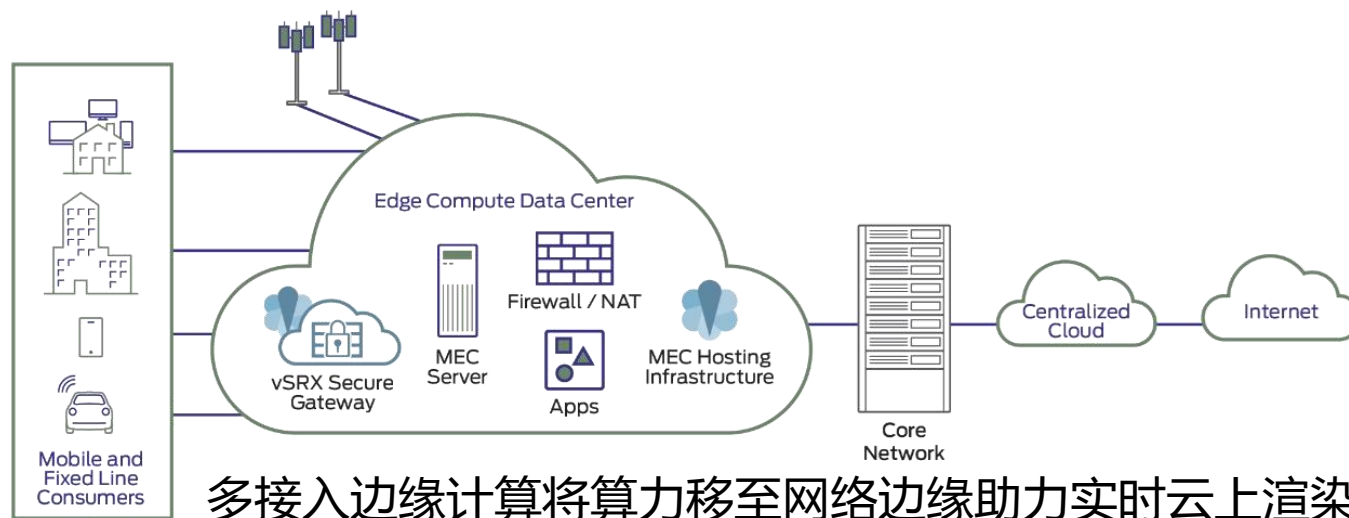
- **云渲染(cloud rendering)**

- 将VR交互应用所须的渲染能力导入云端，有助于降低终端配置成本，帮助用户在移动头显平台获得媲美高价 PC 级的渲染质量。
- 中国信通院发布的《虚拟(增强)现实白皮书(2021年)》明确将云渲染作为VR渲染2.0的关键技术。



VR渲染在哪里计算

- **云渲染(cloud rendering)**
 - 5G技术为用户提供高带宽、低延迟的宽带接入
 - 多接入边缘计算技术(Multi-Access Edge Computing, MEC)通过边侧分析、处理和存储数据，将流量和服务的计算从集中式云转移到网络边缘，以减少延迟并为高带宽应用程序带来实时性能



VR渲染在哪里计算

- **远端渲染(remote rendering)**
 - 将移动设备直接与本地计算机无线连接，既提供了性能，又减小了延迟
 - 目前许多主流VR眼镜已经支持



VIVE的无线升级套件
该套件取代了线缆，为VIVE眼镜提供电源与无线串流信号，可提供2.5小时的体验时长



META QUEST (原Oculus)头显与PC无线连接
Oculus眼镜使用内置的Air Link模块通过Wi-Fi 网络建立与本地电脑的连接。

VR渲染在哪里计算

• 几种渲染方式的对比

渲染方式	优点	缺点
外部计算机渲染	完全无功耗的担忧，延迟与质量都可接受	连杆结构或电缆造成体验不自然，出现绊倒的危险
头显内部渲染	延迟最低	计算能力有限，难以渲染复杂的模型，功耗高
云渲染与远端渲染	功耗较低，渲染质量高	延迟受带宽与信号影响



外部计算机渲染



远端渲染



云渲染

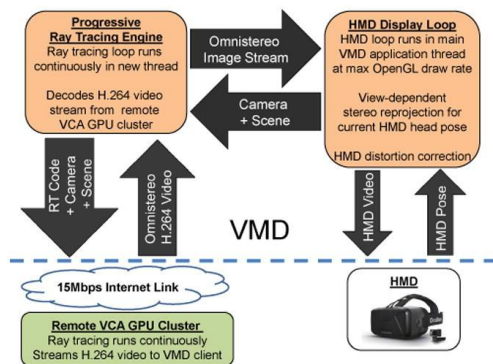
VR渲染方式的发展趋势

图片来源: <https://zerolight.com/news/tech/boundless-xr-the-next-step-to-cloud-rendered-vr>

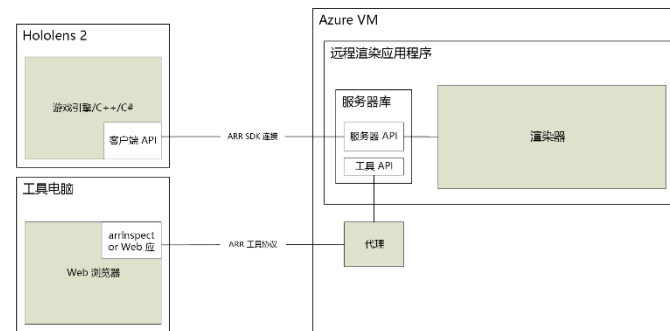
VR渲染在哪里计算

- **混合渲染（端云融合渲染）**

- 头显内部渲染与云渲染相结合
- 通用思路：在高性能云侧运行复杂渲染（复杂光追[1]、复杂模型渲染等[2]），本地运行轻量渲染



使用VCA进行混合渲染



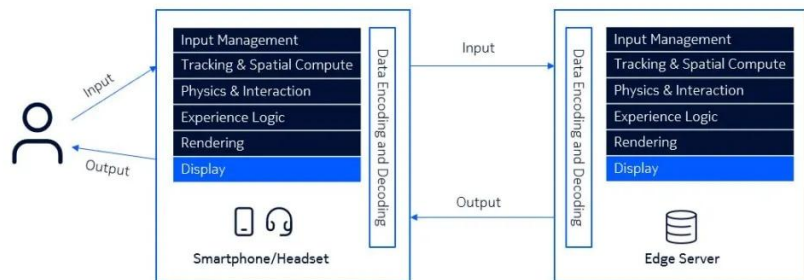
使用微软ARR服务实现混合渲染

[1] Stone, J. E., Sherman, W. R., & Schulten, K. (2016, May). Immersive molecular visualization with omnidirectional ray tracing and remote rendering. In 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW) (pp. 1048-1057). IEEE.

[2] <https://learn.microsoft.com/zh-cn/azure/remote-rendering/overview/about>

端云融合渲染-常见思路

- **协作模式：** 串行分解 & 并行混合
 - 串行：单个任务在时间上切分，顺序经过云和本地
 - 并行：单帧所需的不同任务/元素分别交予云和本地
- **任务划分：** 静态划分 vs 动态（自适应）划分
 - 静态：固定将计算量大、变化少、复杂的元素交给云侧，而将交互频繁、延迟敏感的元素交给本地
 - 动态：考虑实时网络负载、延迟、云侧负载等调整



典型端-云架构示意

<https://www.nokia.com/blog/split-and-conquer-high-quality-xr-for-all/>

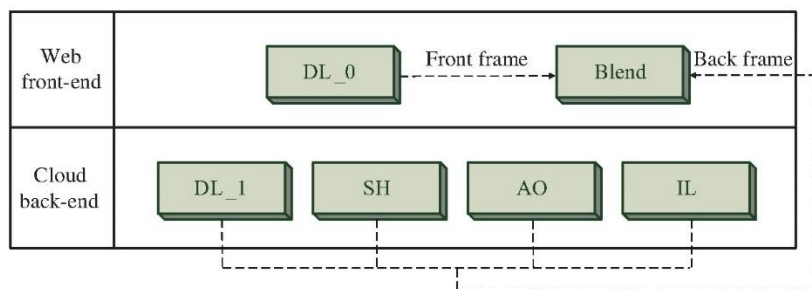
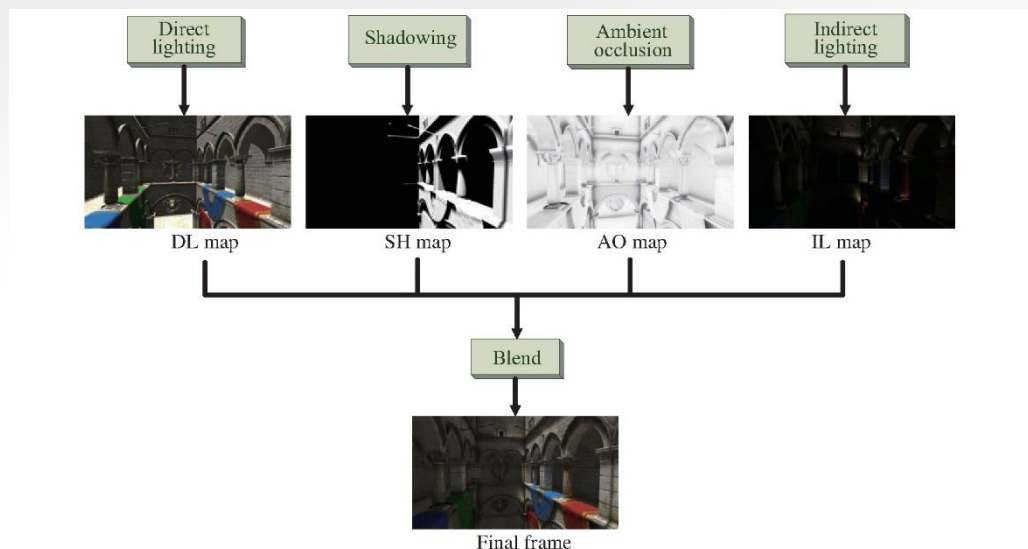


多光源结果并行渲染（左），合成结果（右）

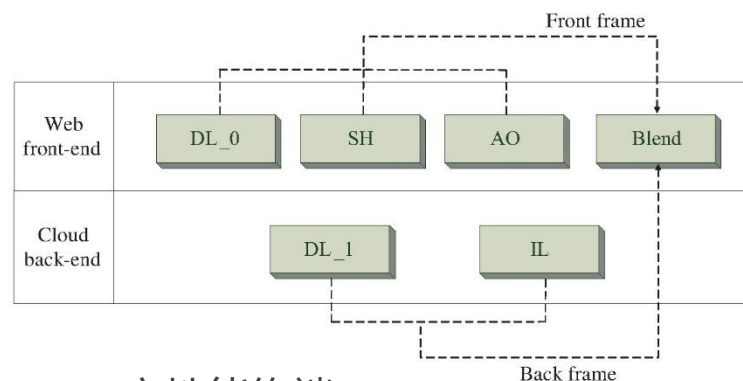
Tan, Yu Wei, et al. "Dhr: Distributed hybrid rendering for metaverse experiences." *Proceedings of the 1st Workshop on Interactive eXtended Reality*. 2022.

端云融合渲染-任务划分案例

- 直接光照
- 阴影
- 环境光遮蔽
- 间接光照



低性能终端：
终端只负责渲染局部光照和最终混合，
云侧负责其他部分



高性能终端：
终端负责更多渲染任务，
云侧仍然负责最复杂的间接光照

端云融合渲染-案例

- 微软 Azure Remote Rendering



1800万面片的模型，
对计算要求很高



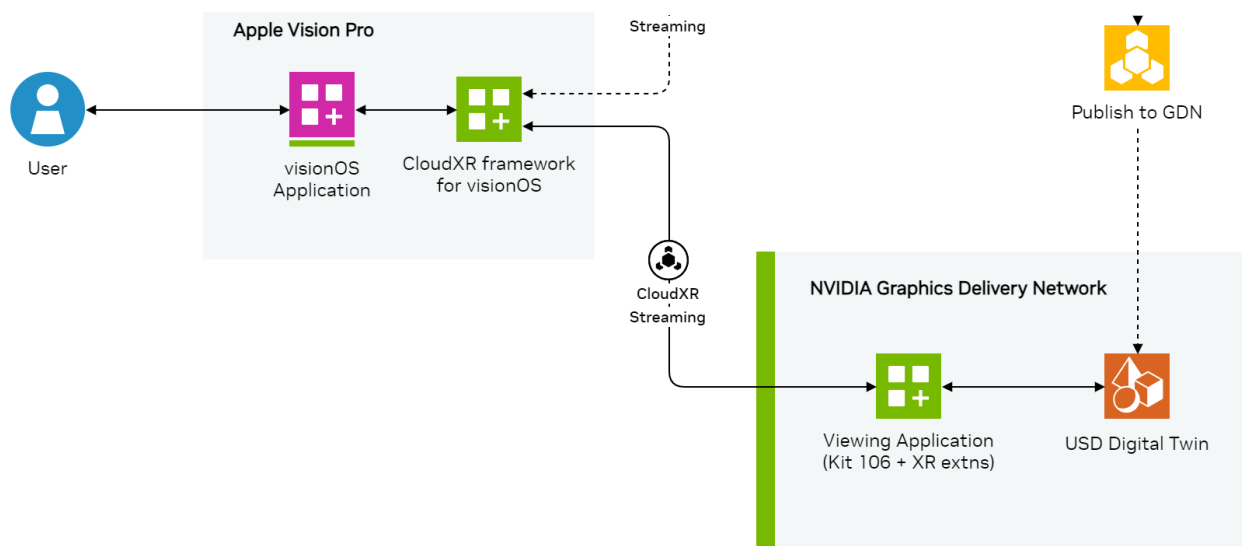
(Hololens 2) 远端渲模型，与本地3D背景合成

<https://learn.microsoft.com/zh-cn/windows/mixed-reality/develop/mixed-reality-cloud-services>

<https://www.youtube.com/watch?v=XR1iaCcZPrU&list=PLk6mkhg-ZCLcetzZLMQ2vxgYe37j31Kck>

端云融合渲染-案例

- **NVIDIA Omniverse + Apple Vision Pro**
 - 云侧 “Viewing Application” 用高性能RTX GPU渲染
 - 端侧visionOS Application负责用户UI和实时交互处理，CloudXR framework负责接受视频流，两者在设备上合成



想一想

- 云渲染对实现元宇宙有何意义？

- 云渲染平台有望实现对元宇宙“虚拟世界”的重定义。.....由于人机交互程度提升、终端轻量化等因素，未来元宇宙计算处理与显示输出将存在一定程度分离。终端主要承担光学、显示、环境和动作的感知，实时渲染和空间计算将通过超宽带低时延可靠网络，传送到云端进行处理。——《元宇宙白皮书（2023年）》



用户利用5G网络游玩云上VR (以HTC 5G Hub产品为例) (动图)

目录

01 VR渲染引言

02 视觉渲染基本方法

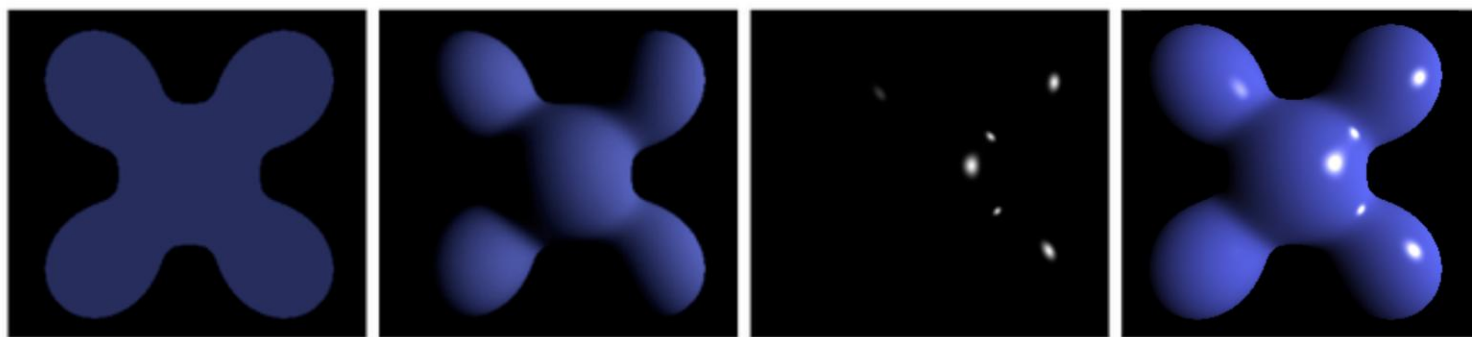
03 针对VR的视觉渲染增强

04 VR中的其他感官渲染

视觉渲染

- 基本渲染模型

- 局部：Phong模型



Ambient

+

Diffuse

+

Specular

=

Blinn-Phong
Reflection

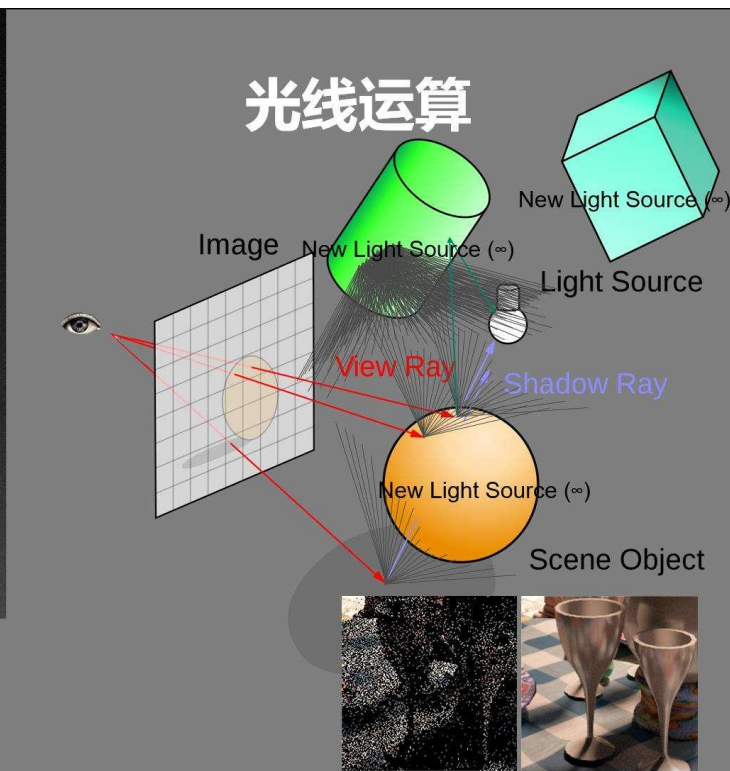
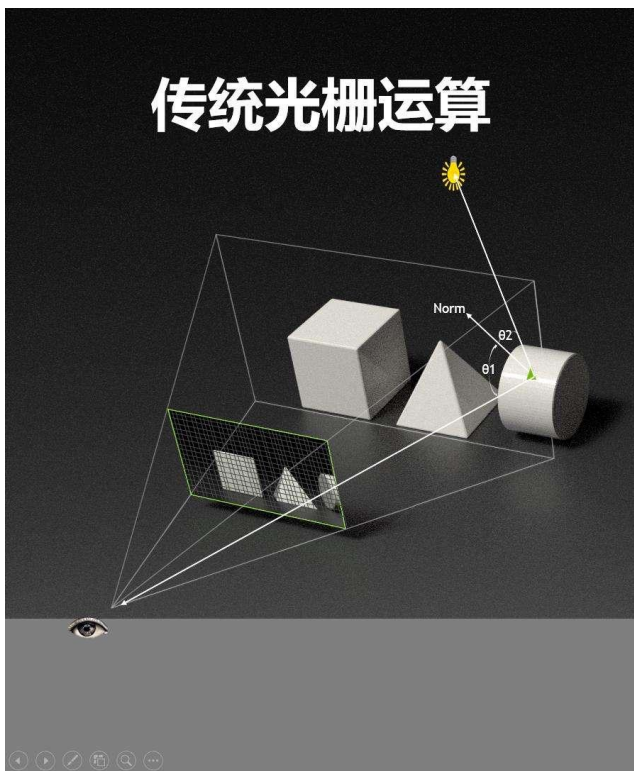
$$L = L_a + L_d + L_s$$

$$= k_a I_a + k_d (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

视觉渲染

- **基本渲染模型**

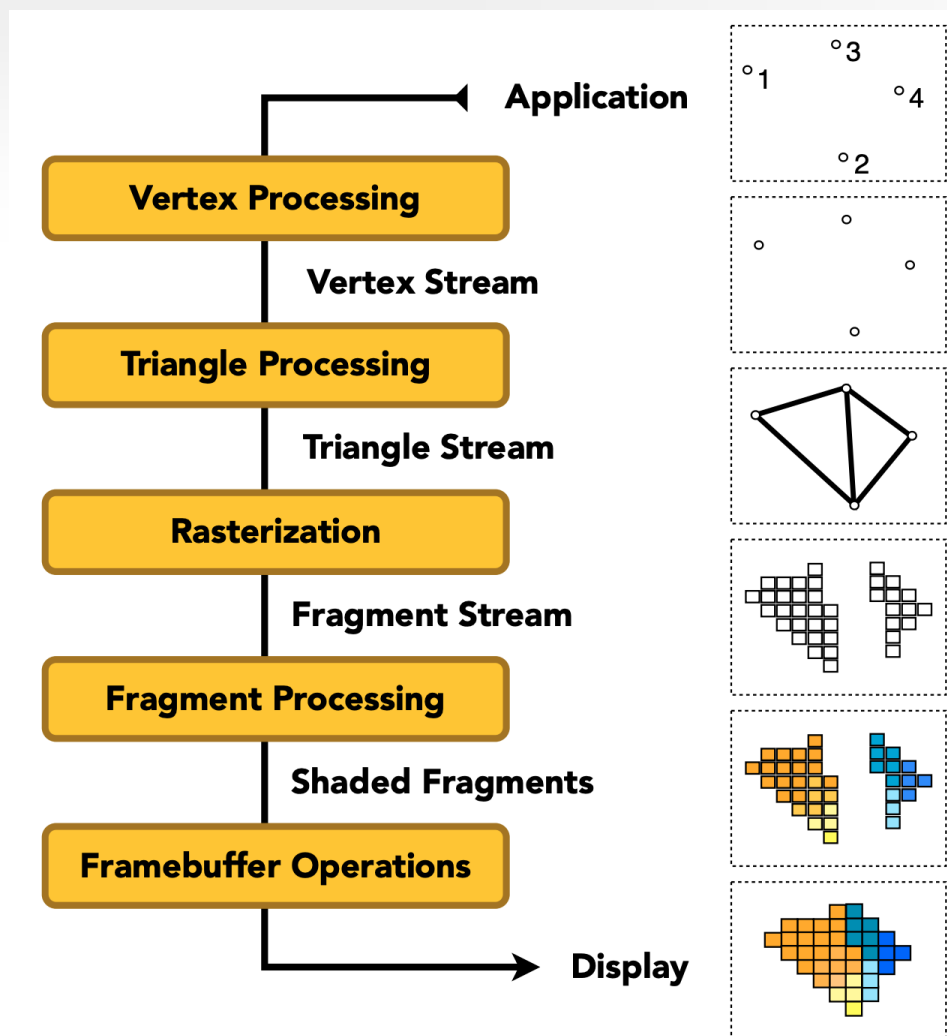
- 局部：Phong模型
- 全局：光线跟踪



视觉渲染

- **基本渲染模型**

- 局部: Phong模型
- 全局: 光线跟踪
- GPU渲染通路

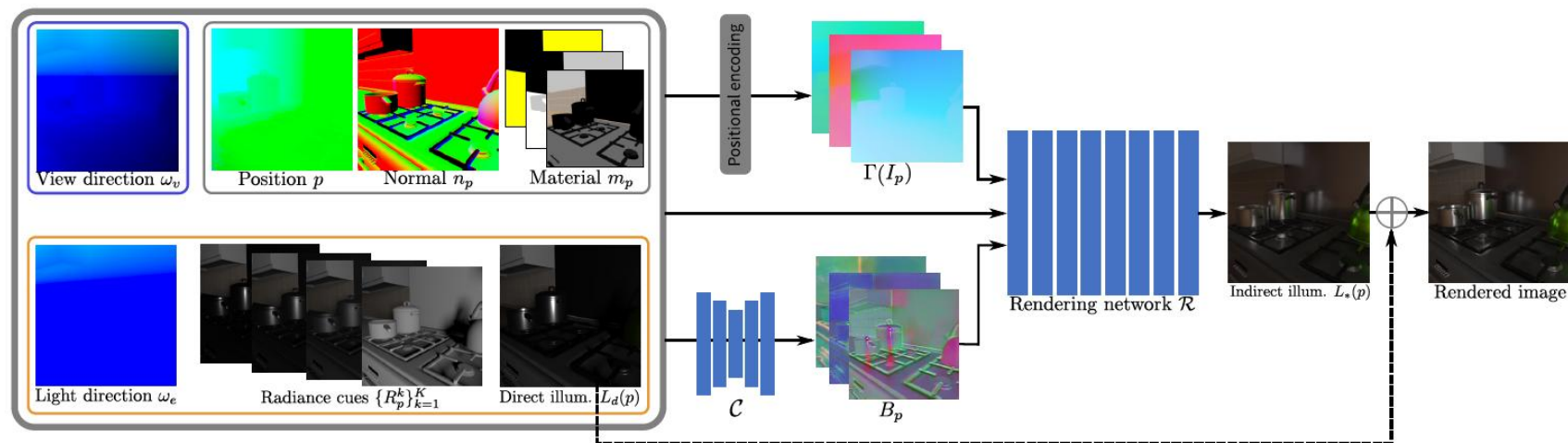


视觉渲染

• 基本渲染模型

- 局部: Phong模型
- 全局: 光线跟踪
- GPU渲染通路

• 神经渲染



目录

01 VR渲染引言

02 视觉渲染基本方法

03 针对VR的视觉渲染增强

04 VR中的其他感官渲染

针对VR的视觉渲染增强

- **时间扭曲与异步时间扭曲**：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

VR渲染延迟

- 什么是**VR渲染延迟**?
 - 从用户运动到用户看到响应的时间被称为动显延迟 (Motion-to-photons latency)
 - 例：AR延迟包括相机采集-传输-处理-渲染-传输-上屏
- Oculus VR 的首席科学家Michael Abrash：人可接受的动显延迟 $\leq 20\text{ms}$ ，理想的延迟 $\leq 7\text{ms}$ [1]。
- 虚拟现实专家Jason Jerald也曾建议最佳动显延迟仅为**3ms** [2]。



Michael Abrash
计算机图形学界泰斗
Oculus VR 首席科学家



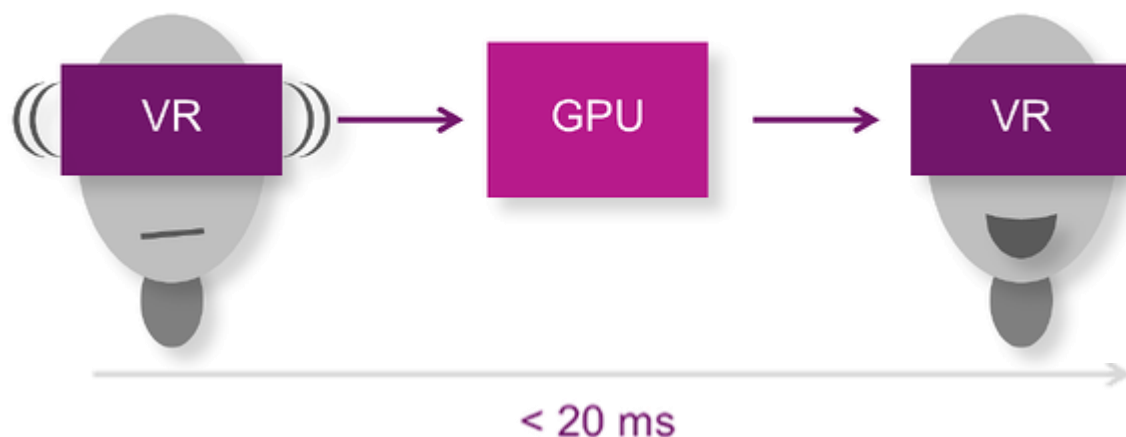
Jason Jerald
NextGen Interactions
首席顾问

[1] <http://blogs.valvesoftware.com/abrash/latencythe-sine-quan-non-of-ar-and-vr/>

[2] Jerald, J. J. (2009). Scene-motion-and latency-perception thresholds for head-mounted displays (Doctoral dissertation, The University of North Carolina at Chapel Hill).

VR渲染延迟：解决思路

- 加快数据传输：尽可能快地将信息从输入和渲染设备传输至头显
- 加快渲染速度：减少渲染图像所需的时间
- 然而，解决渲染延迟并非仅能依靠硬件提升，**通过软件方法依然可以在有限硬件条件下降低延迟**



<https://blog.imaginationtech.com/presence-in-virtual-reality-and-how-powervr-gpus-can-help/>

加快数据传输和渲染速度是解决渲染延迟的两个基本方法

时间扭曲(Timewarp)

- **问题：** GPU根据头显位置渲染画面并传输到头显的过程中，用户头部位置发生了变化，导致观看的画面滞后。严重时可能产生晕动症。



- 如何避免**物理移动导致的渲染延迟**？一种解决方法是通过采集陀螺仪数据，预测出下一帧呈现时用户头部应有的旋转和位置，**按照这个预测位置渲染画面并传输给用户。**
- 这种根据跟踪数据重新调整渲染图像的渲染方法叫作**时间扭曲** (Time warping或Timewarp)。

时间扭曲

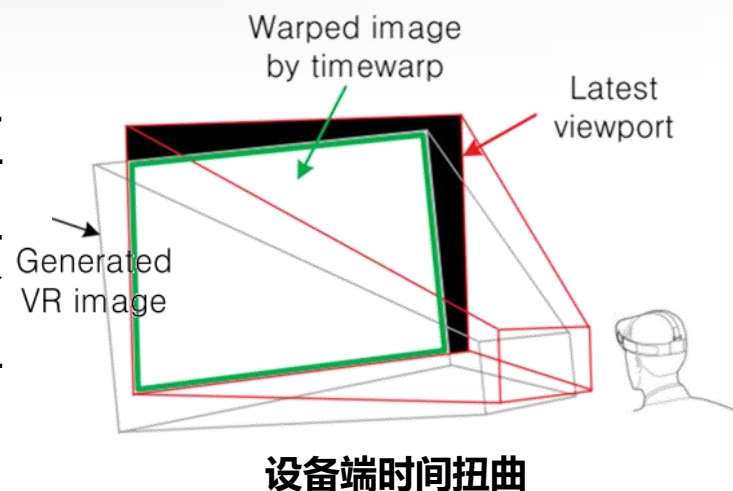
- **渲染端时间扭曲(renderer-side timewarp)**
 - GPU根据渲染后的头部方向来渲染图像，并将其发送到用户设备。
 - 适用于GPU在本地并通过电缆直接与用户连接的情况。由于电缆传输耗时可以忽略不计，渲染延迟可以由GPU内在帧率准确估计，因此可以准确渲染所需视图。
 - 但是如果GPU在云端或网络边缘时，由于**传输耗时抖动**，渲染端时间扭曲就会产生**视图不一致**的问题。此时需要**设备端时间扭曲**来解决。

时间扭曲

- **设备端时间扭曲(device-side timewarp)**

- 设备端时间扭曲与渲染端时间扭曲的基本原理相同，也是根据最新头部位姿调整图像。不同的是其时间扭曲发生在**头显端**，头显对收到的图像根据最新姿态做几何扭曲再呈现给用户。

- 设备端时间扭曲可以使图像更精确地匹配用户最新位姿，但由于图像经过后期加工，比渲染端时间扭曲会引入更多**失真**。



*为了减少warping带来的失真，GPU一般会渲染一个比需求更大的视图，然后利用额外的图像边缘来适应视角切换和头部转动。

时间扭曲的发展

- 时间扭曲的思想在VR领域已存在多年[1]。
- 2014年4月，美国著名电子游戏工程师、Oculus VR首席技术官 John Carmack 将这一功能添加进 Oculus Rift SDK v0.3.1 中。
- 而在更早的2013年2月，Carmack就在博客中详细描述了时间扭曲的想法[2]。
- 如今，所有的主流VR平台都已经使用时间扭曲来降低延迟，时间扭曲已成为VR渲染的默认标配。



John Carmack
id Software创始人
曾任Oculus VR首席技术官

[1] Smit, F. A., van Liere, R., & Fröhlich, B. (2008, October). An image-warping vr-architecture: design, implementation and applications. In Proceedings of the 2008 ACM symposium on Virtual reality software and technology (pp. 115-122).

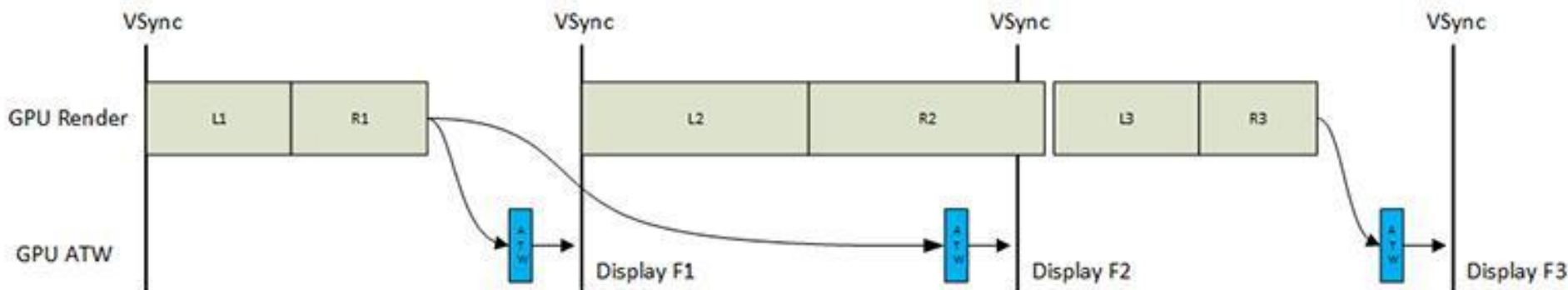
[2] <https://web.archive.org/web/20140719085135/http://altdev.co/2013/02/22/latency-mitigation-strategies/>

异步时间扭曲(Asynchronous Timewarp, ATW)

- 标准时间扭曲只是降低了用户感觉到的延迟，**但没有真正提高帧率！**
- 帧率对VR体验非常重要，我们知道一个比较良好的VR体验帧率至少要达到60帧甚至90帧每秒。
- **异步时间扭曲**采用了和**设备端时间扭曲**相同的几何扭曲概念，**但用来补偿缺失的帧。**
- **异步时间扭曲**的基本原理是，如果当前帧没有及时完成渲染，使用最新的位姿跟踪数据重新投影前一帧，并将它展现给用户。

异步时间扭曲

- **异步**：利用上一帧的“插帧”可以与渲染并行发生，而不是在渲染之后发生。在知道真实帧是否会按时完成渲染之前，合成帧就已经准备就绪。

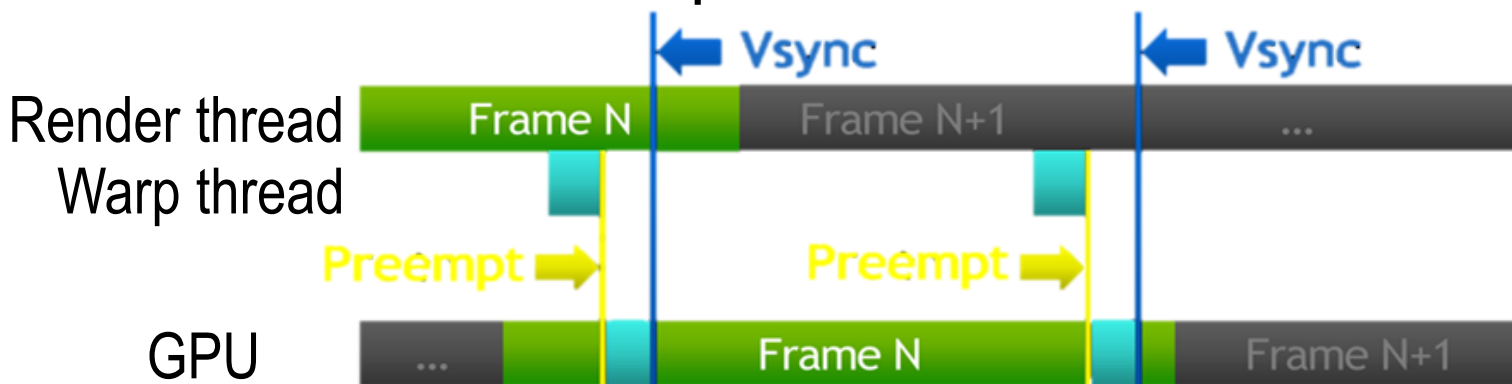


ATW处理过程：异步地利用上一帧合成当前帧，以便当前帧没及时渲染好时呈现给用户

- **ATW如何帮助稳定帧率：**如图，第一帧时画面渲染及时完成，直接显示；第二帧右眼渲染未能及时完成。此时若没有ATW则会画面抖动，而有ATW就能将第一帧调出来，加上头显运动变化后重新显示，从而保持帧率。

ATW对VR系统的意义

- ATW把VR渲染管线变为两个线程，渲染线程和Timewarp线程，两者异步工作。渲染线程按GPU的帧率生产足够多的图像帧，而Timewarp线程则把渲染帧做Timewarp后贴在屏幕上。
- 这样一来，无论GPU帧率是多少，**ATW始终可以保证头显的帧率为固定值**（如90fps），做到与GPU帧率无关。

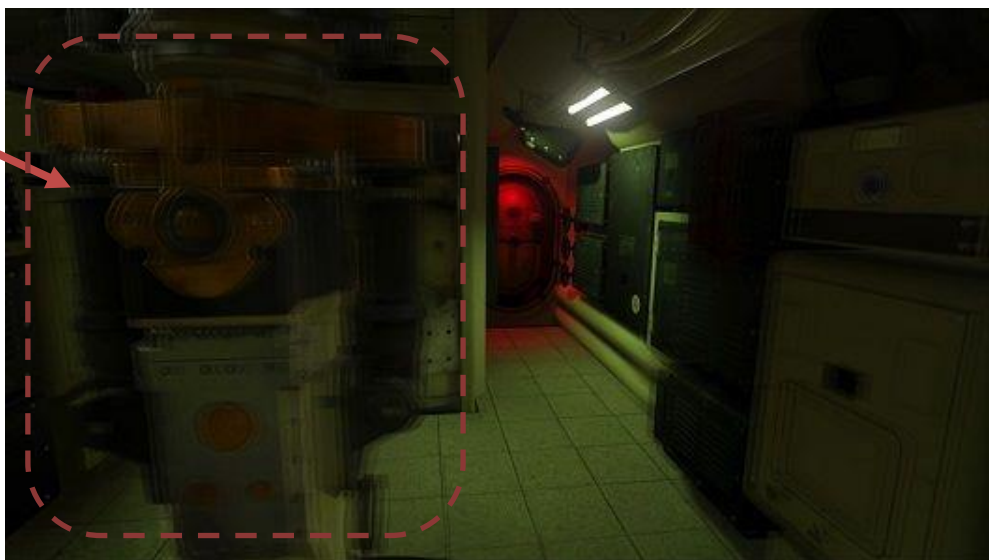


NVIDIA游戏开发者大会(GDC) 2015上展示的ATW特性

ATW的局限

- **局限一：位置抖动**
- 如果用户头部在旋转运动之余产生了**平移**分量，则场景中的物体由于远近不同就会产生**视差**。这种视差无法通过图像warping来消除。从用户感受来说，采用ATW的VR系统中用户就会因头部平移而看到近处物体有多个抖动的像。

Positional Judder

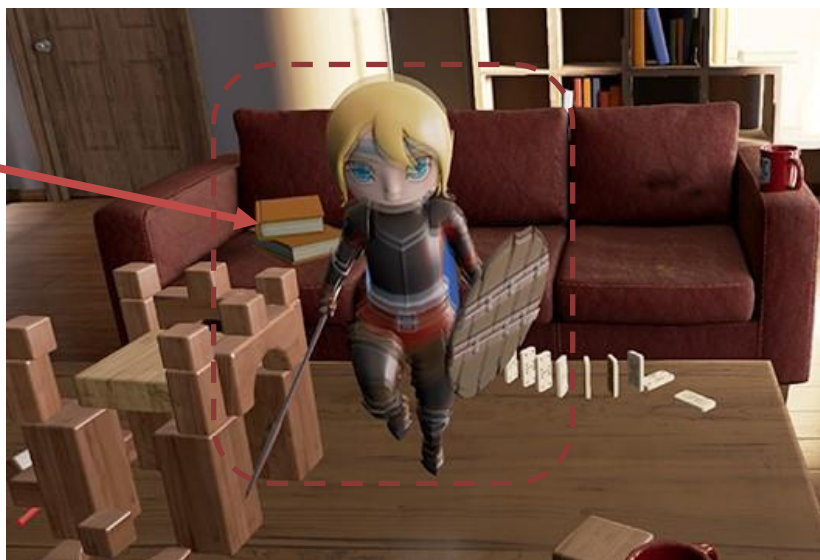


左图：由于视差原因ATW导致近处物体位置抖动

ATW的局限

- **局限二：移动和动画物体抖动**
- 相似地，对于ATW 生成的帧，由于其不知道虚拟物体的移动和动画，当与渲染的帧交错呈现给用户时，用户就可以观察到移动的虚拟物体产生伪影。伪影严重程度取决于场景中移动对象的数量、投影面积和速度。

Judder



左图：由于物体移动产生的伪影

针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- **异步空间扭曲**：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

异步空间扭曲(Asynchronous Spacewarp, ASW)

- 鉴于ATW仅能在头部旋转条件下较好地生成缺失帧，无法处理头部平移和场景物体移动所产生的伪影，2016年12月Oculus发布了**异步空间扭曲** (ASW) 来更好地合成缺失帧。
- ASW本质是一种外推算法，通过先前帧之间的差异（即运动）来估计下一帧。

GPU仅按照45fps
的帧率渲染



使用ASW以90fps
的帧率呈现给用户

不使用 / 使用 ASW 的 VR 渲染效果对比 (动图)

ASW的原理

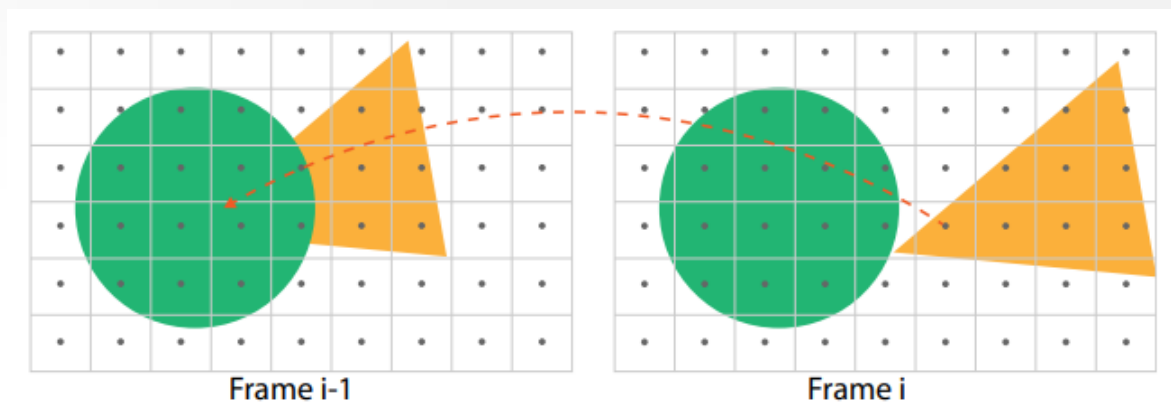
- 获取由GPU渲染的应用程序的前两个帧，计算差异，然后根据空间变换、用户当前的头部位置和运动创建一个所需“合成帧”。



- ASW允许VR游戏以每秒 45 帧的低帧率运行，并在其间创建合成帧，从而在头显中输出每秒 90 帧的流畅的画面。

ASW如何处理物体运动

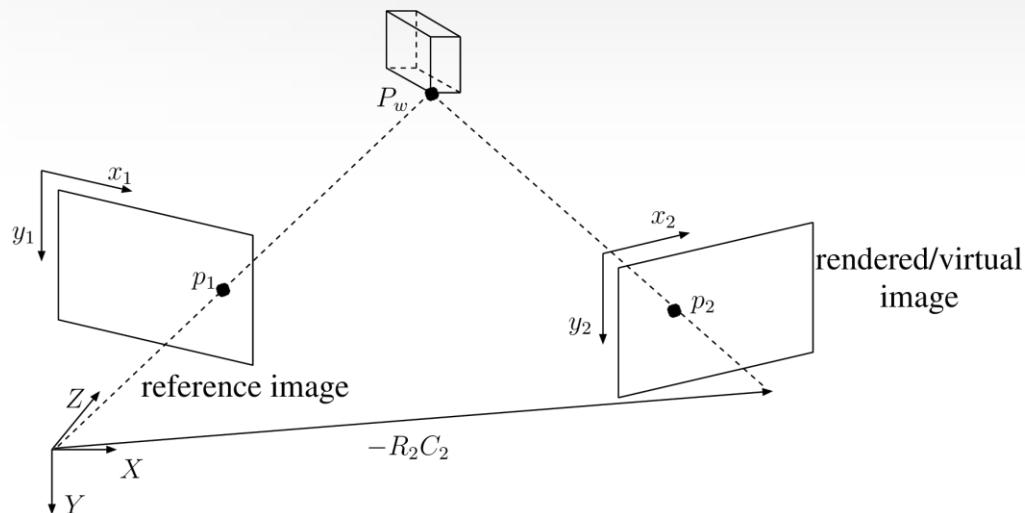
- 借助**运动矢量** (Motion Vector)



- 运动矢量描述了空间中某一点在不同时刻之间的位移
- 在VR渲染中，物体的运动情况是已知的。将物体在三维空间中的位移投影到屏幕空间中，即可得到空间扭曲所需的运动矢量
- 根据前两帧间的运动矢量，可以估计出外插帧内物的位置

ASW如何处理头部平移带来场景变化

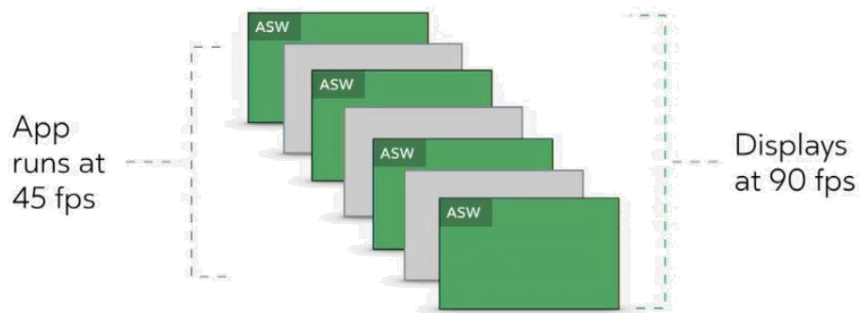
- 基于深度 (Depth) 的扭曲



- 对于前一帧上的像素点 P ，根据该帧的相机参数和 P 的深度，可以得到 P 点的三维空间坐标。若 P 点静止，即可根据外插帧的相机参数，将 P 点投影到屏幕空间上

ASW与ATW的关系

- 尽管ASW是ATW的“加强版”，但由于其计算花销大于ATW，并不能取代ATW。ATW由于成本低、速度快，可以始终处于活动状态；而ASW则在需要时启用。
- 在SteamVR的Interleaved Reprojection技术中（现升级为“Motion Smoothing”技术），仅当应用程序在几秒钟内持续丢掉多帧时，ASW才会自动启用，它强制应用程序以半帧速率 (45fps) 运行，并每隔一帧合成一次生成帧。



针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- **多路复用渲染**：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

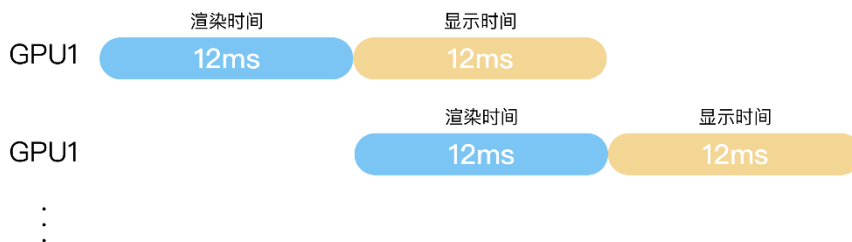
多路复用渲染(Multi-GPU rendering)

- **什么是多路复用渲染/多GPU渲染**
- 在具有多 GPU 的计算环境中，将渲染任务分配给不同的 GPU以加快图像渲染速度的方法。
- VR渲染中常见的多路复用渲染方式可以分为：
 - **交替渲染法**：使用多个GPU交替渲染帧。
 - **任务分解法**：在不同GPU之间划分任务，将每个GPU的子任务结果重新组合成一个整体。

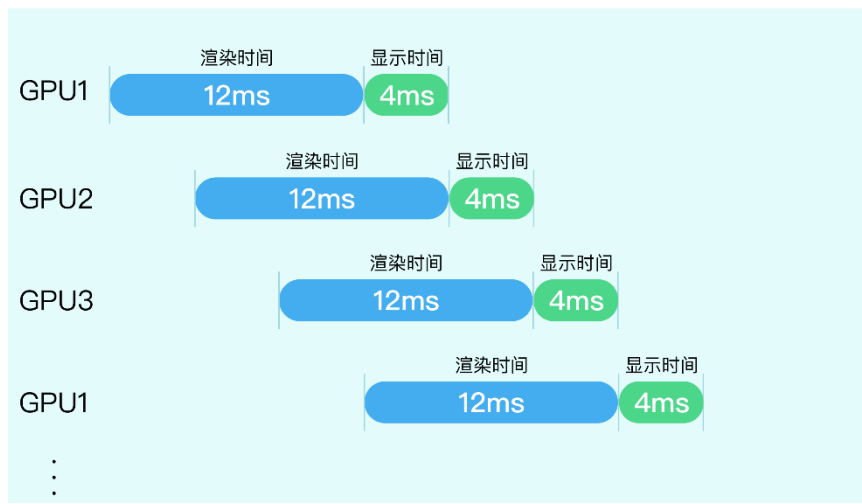
多路复用渲染——交替渲染法

- 用**多个GPU渲染交替帧**，是最常用的多路复用渲染方式。
- 比如在两个 GPU 的情况下，一个渲染偶数帧，另一个渲染奇数帧。GPU启动时间错开以向头显稳定输出帧率。

单GPU渲染

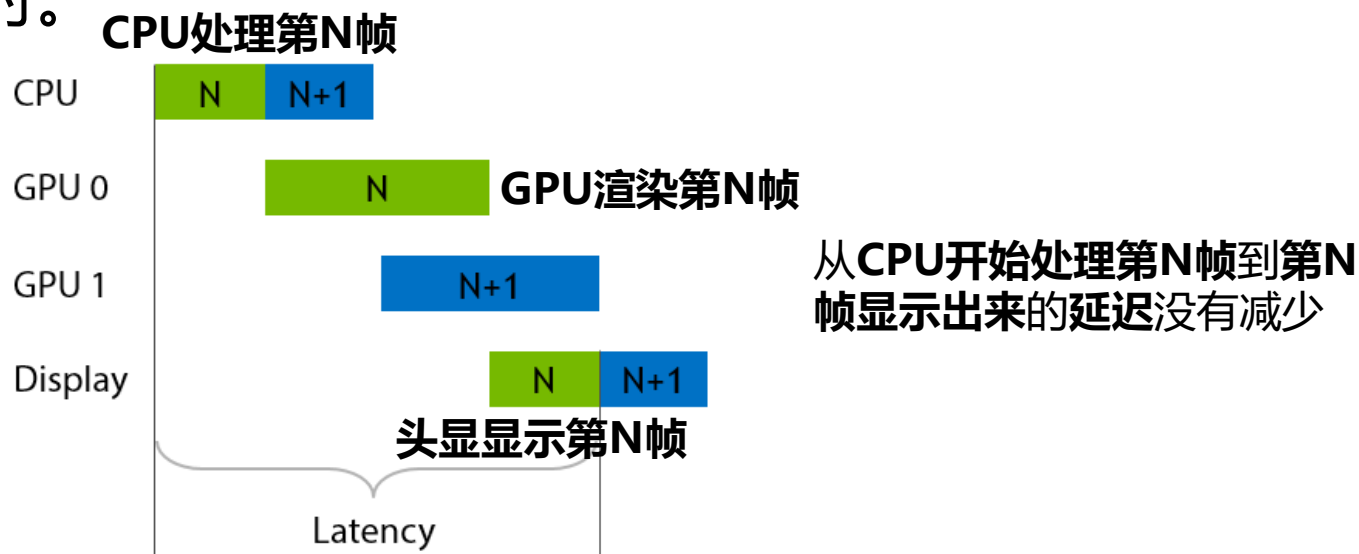


多GPU交替渲染



多路复用渲染——交替渲染法

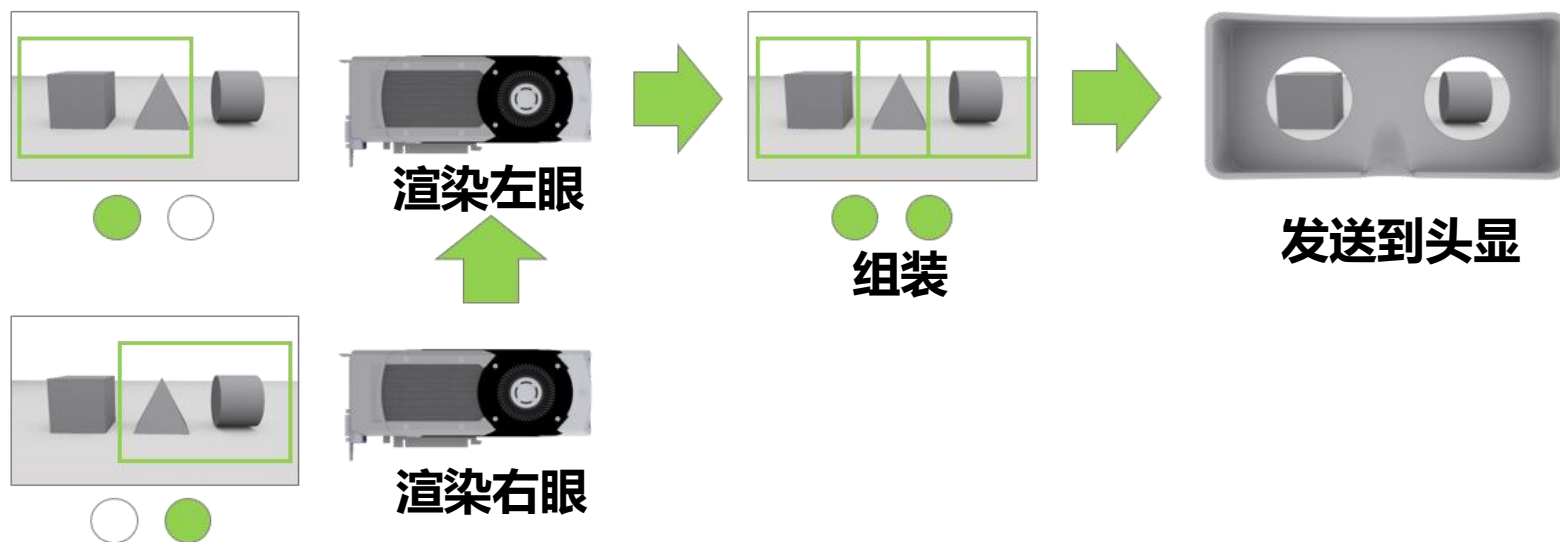
- 交替渲染技术按时间维度而不是任务维度执行多路复用渲染，虽然可以**提高帧率**，但**不会减少系统的最小延迟**。
- 为何交替渲染无法减少系统最小延迟？因为对一帧来讲，从系统调用渲染到渲染完毕的延迟依然为单个GPU渲染一帧的耗时。



交替渲染法可以**提高帧率**但无法降低最小延迟

多路复用渲染——任务分解法

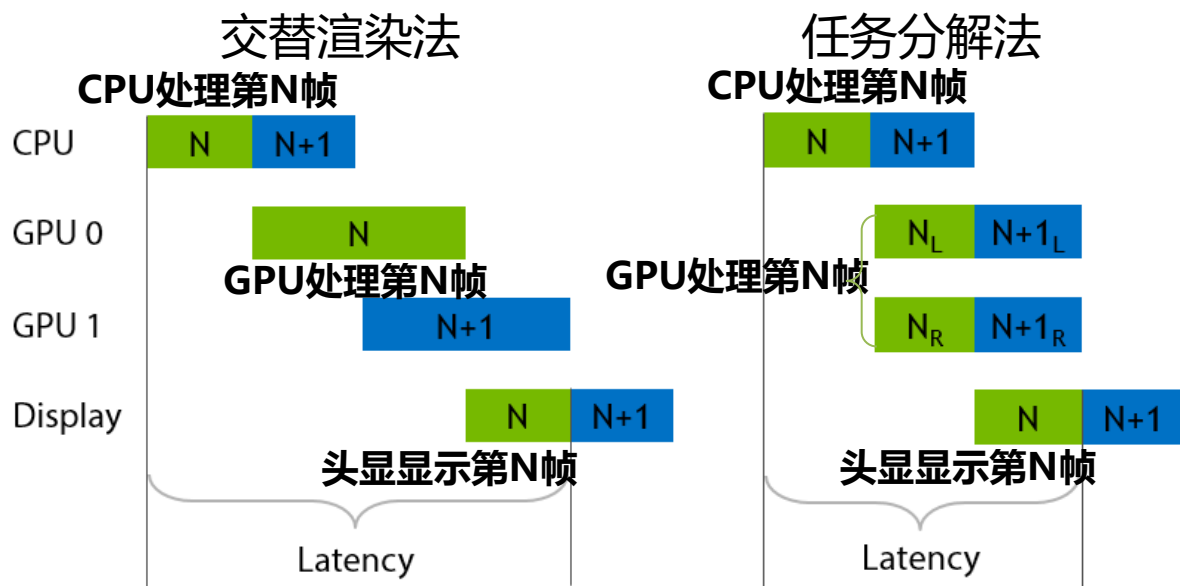
- 将绘制**单个帧的工作分摊到多个GPU之间**，将每个GPU的子任务结果重新组合成一个整体，可以降低渲染一帧的时间，**减少系统最小延迟**。
- 鉴于两眼视图彼此独立，从而可以为2个 GPU 各分配一眼以加速渲染，然后将两个图像组合成一个帧发送到头显。



多路复用渲染

• 交替渲染法与任务分解法的优劣

多路复用渲染	帧率	最小延迟	额外通信
交替渲染法	提升	没有减小	不需要
任务分解法	提升	减小	需要



交替渲染法与任务分解法中各设备处理第N帧的时间

针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- **重投影技术**：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

渲染调用中CPU的重复工作

- 在一次渲染调用中用GPU对视图进行着色之前，还要**事先用CPU对场景进行诸多处理**（物体排序、阴影计算等）。
- 人眼的瞳距仅几厘米，在大多数场景中针对双眼的物体排序的结果是相同的。一些场景因素如灯光投射阴影的位置对两眼来说也是一样的。

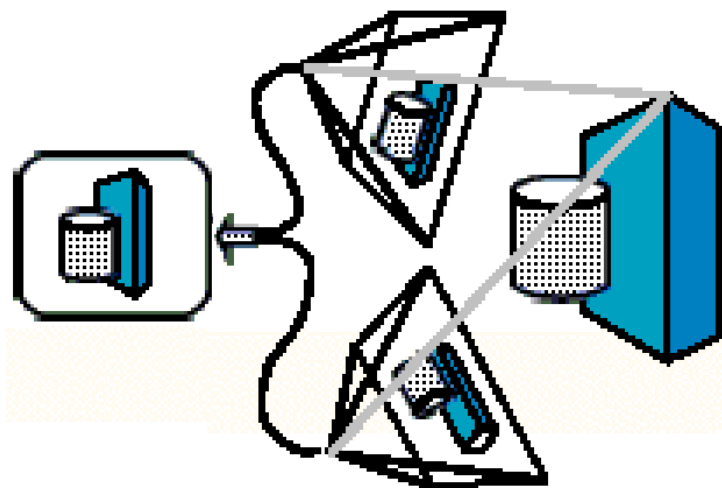


左图：人眼双目图像具有很大的相似性，重复渲染调用增加了额外的工作。

- 因此如果对每个视图都进行一次完整的渲染调用，则**大大加重了CPU的重复工作**。

重投影技术：分离视图法

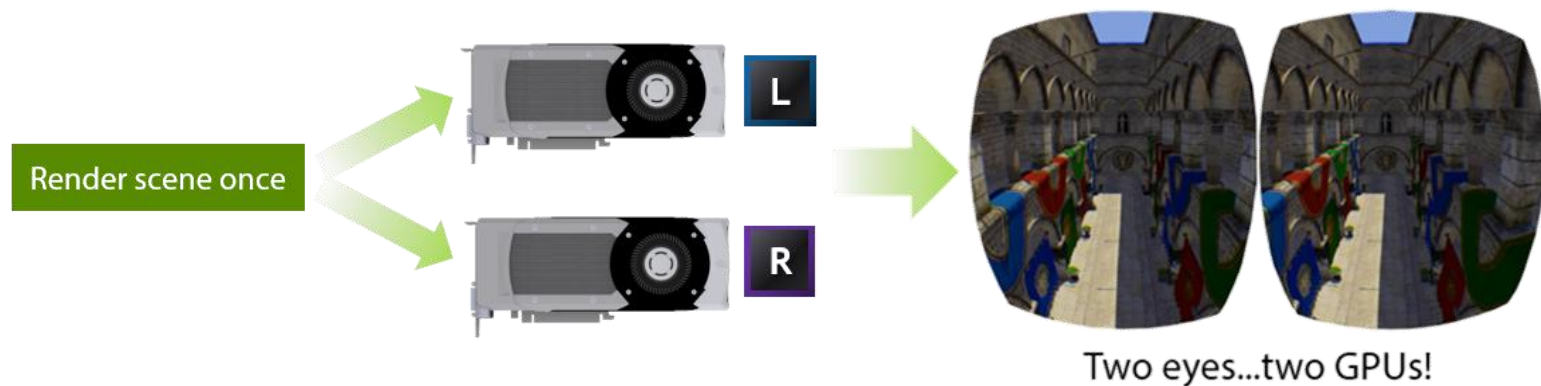
- 只渲染两只眼睛中心位置的视图（仅一次渲染调用），然后使用**基于图像的渲染（Image-Based Rendering, IBR）**方法重新投影图像并填补空白得到每只眼睛的图像[1]，以节约渲染开销。



示意图：通过重投影来分离视图，仅一次渲染调用便可得到双目图像

重投影技术：分割渲染法

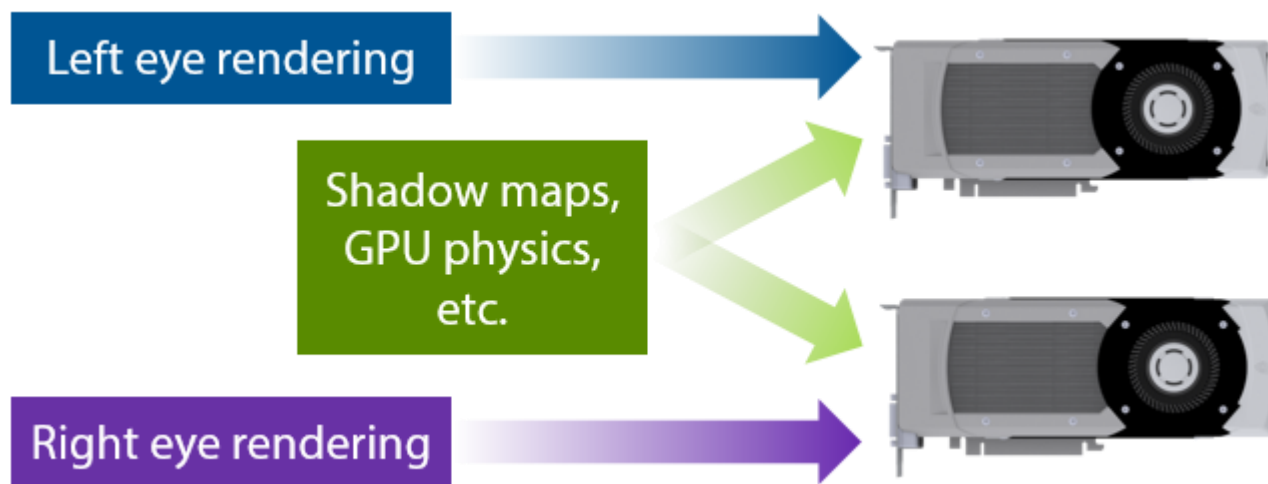
- 在渲染阶段，两只眼睛的绘制调用几乎完全相同（双眼可以看到相同的对象，渲染相同的几何体，使用相同的着色器、纹理等等），两眼绘制的唯一区别是它们的视图位置——视图矩阵中的几个数字。
- 因此只需**一次渲染调用**，执行一次排序、剔除等类似操作，再通过**重新投影**分割渲染两眼视图，就可以节省计算。



通过一次渲染调用分割渲染两眼视图

重投影技术：分割渲染法

- 如今主流游戏引擎和 VR 渲染系统中支持一个专门的渲染路径，可以一次性完成所有的非着色工作，然后为左右眼睛做轻微的调整以便正确地渲染它们，如NVIDIA **VR SLI** 技术。



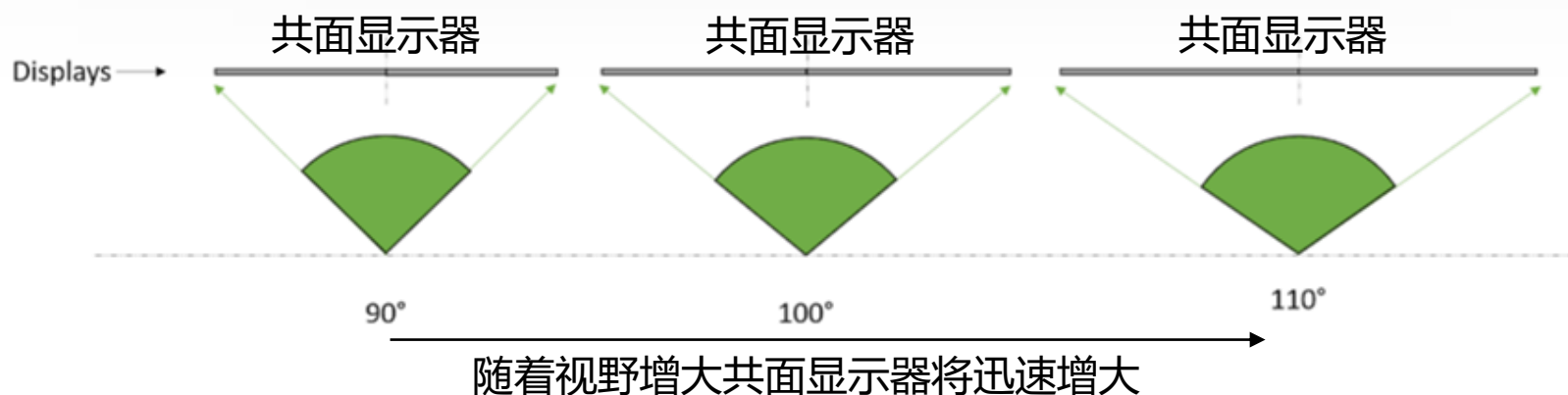
NVIDIA **VR SLI** 技术可通过一次渲染调用分割渲染两眼视图，同时几乎不增加CPU的额外开销

针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- **多视图渲染**：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

共面显示器的视野问题

- **头显视野(Field Of View, FOV)**: 视野越大, 体验就越身临其境。增加视野已成为VR头显发展的趋势。



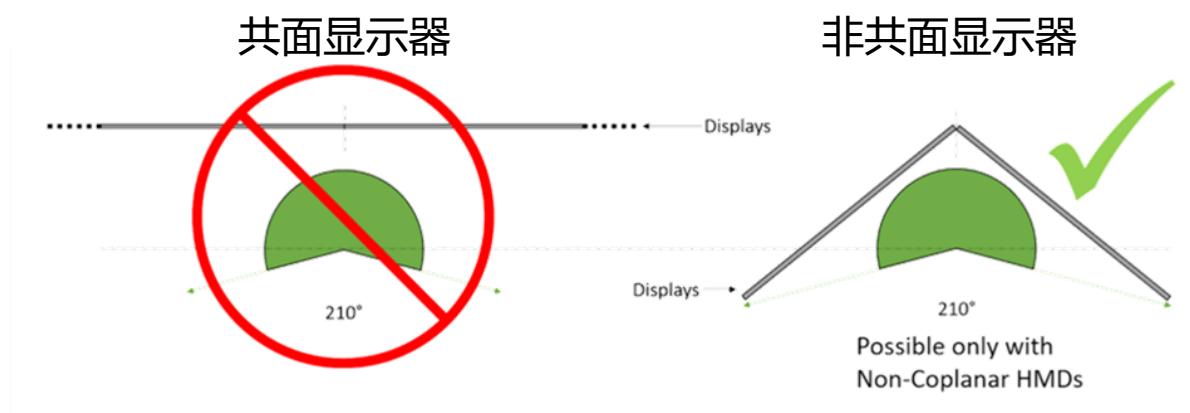
- 随着 FOV 的增加, 共面显示器尺寸会迅速增加, 头显变得笨重; 在110° FOV 以上, **共面显示器将会大得难以承受。**

右图: 随着FOV增大, 共面显示器将使头显变得笨重且不符合人体工程学



非共面显示器

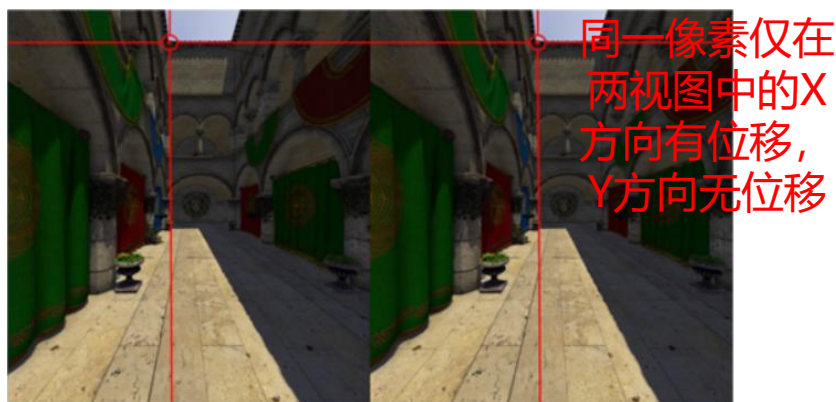
- **非共面显示器**(Non-coplanar HMDs): 因为人类现实中会注意到超过 180° FOV 中的物体和运动, $180^\circ + \text{FOV}$ 可以让 VR 更加逼真。但共面显示器不可能实现这样的超宽 FOV。此时只能在头显中引入**非共面显示器来扩大FOV**。



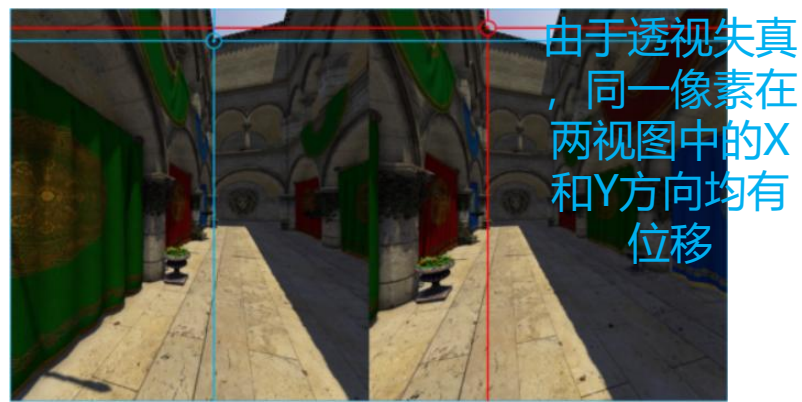
使用非共面显示器来实现超过 180° 的视角

非共面显示器中的透视失真

- 渲染一个 3D 视图时，渲染引擎在 3D 空间中创建虚拟相机，根据相机位置对场景中的几何体执行计算，然后执行像素着色。典型的VR头显仅需从两个方向平行、位置稍微偏移的相机处渲染同一场景的两个视图，并投射到头显左右眼。**非共面显示器意味着双眼的相机不再平行。**由于显示器的不同角度导致按原先的渲染方式将产生**透视失真**。



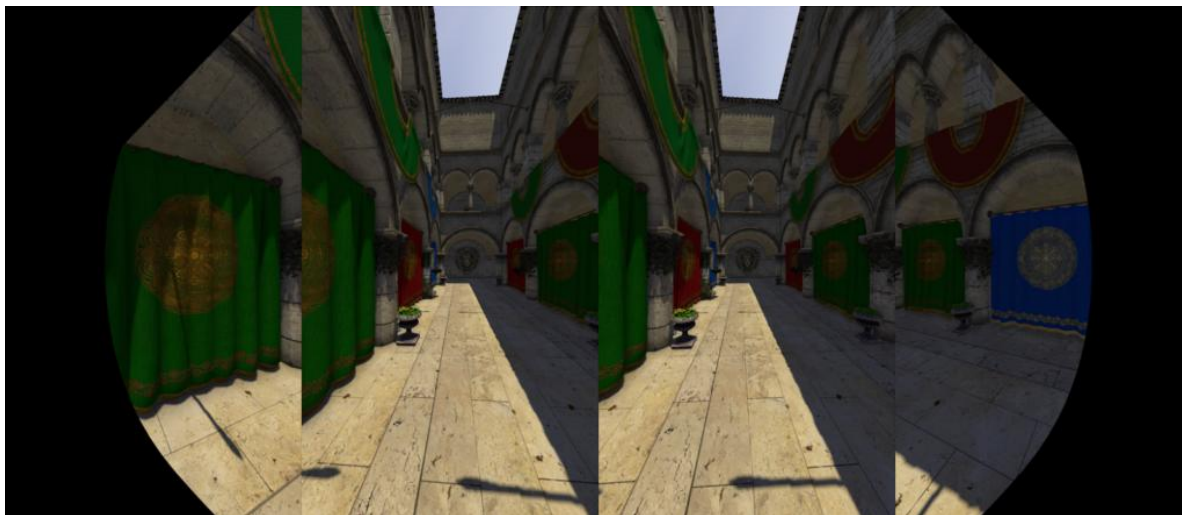
典型的VR头显仅需从两个平行的相机位置渲染场景的两个视图



传统的双目渲染方式将在非共面头显中产生**透视失真**现象

多视图渲染 (Multi View Rendering, MVR)

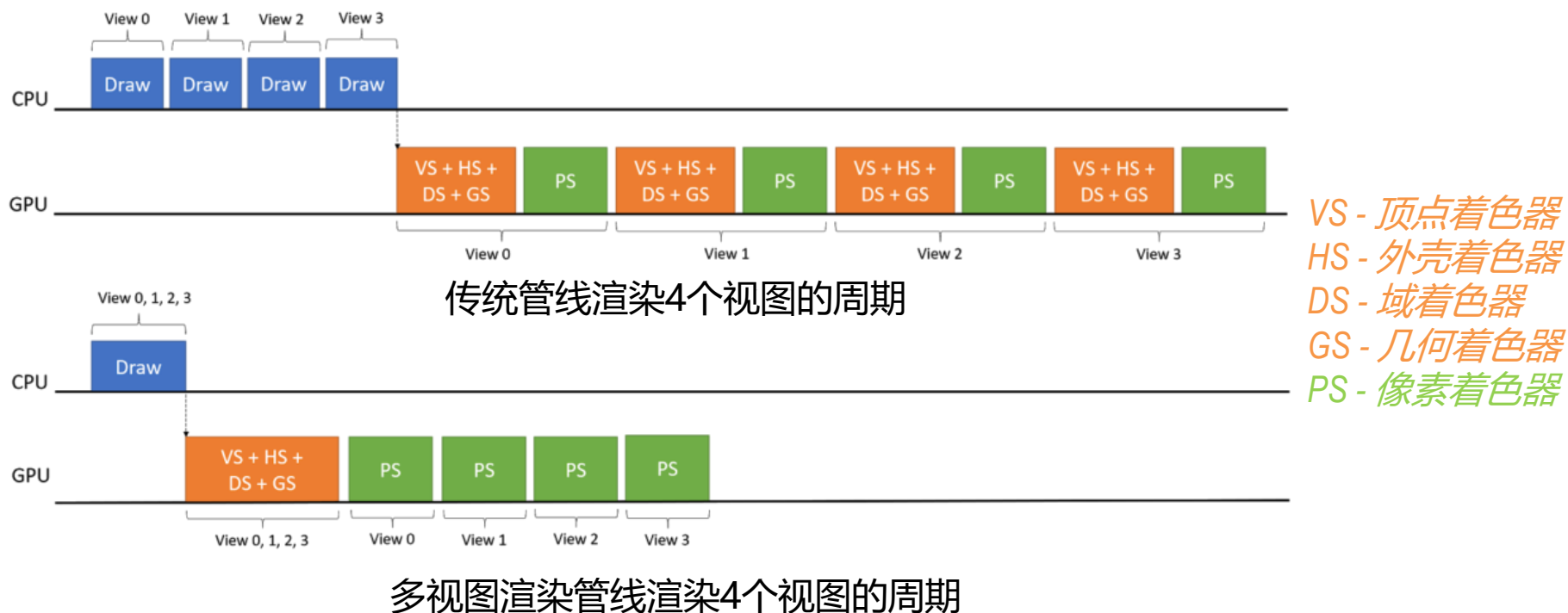
- 为了减小非平面显示器中的超广角显示内容的透视失真，同时保证视觉细节，引入了**多视图渲染**的解决方案：
- 将超宽视图中两个显示器中每一个进一步分成两个视图。
- 场景的每个部分都可以以良好的像素密度进行渲染，即使在屏幕边缘也能保留细节。



支持超光视野的多视图渲染

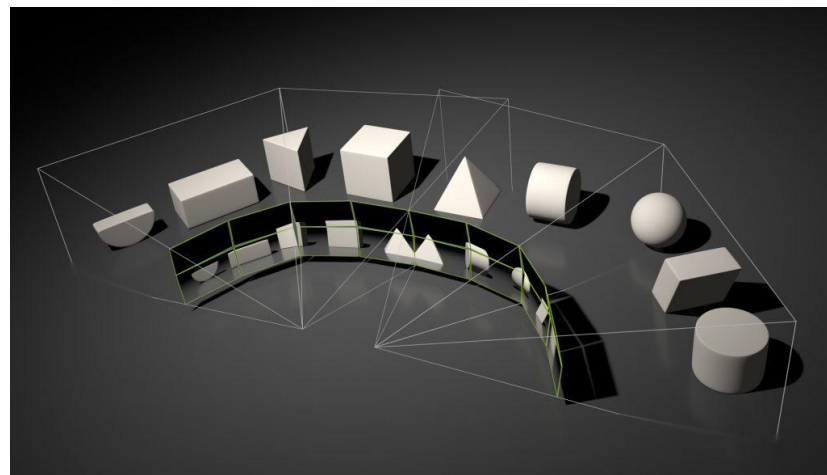
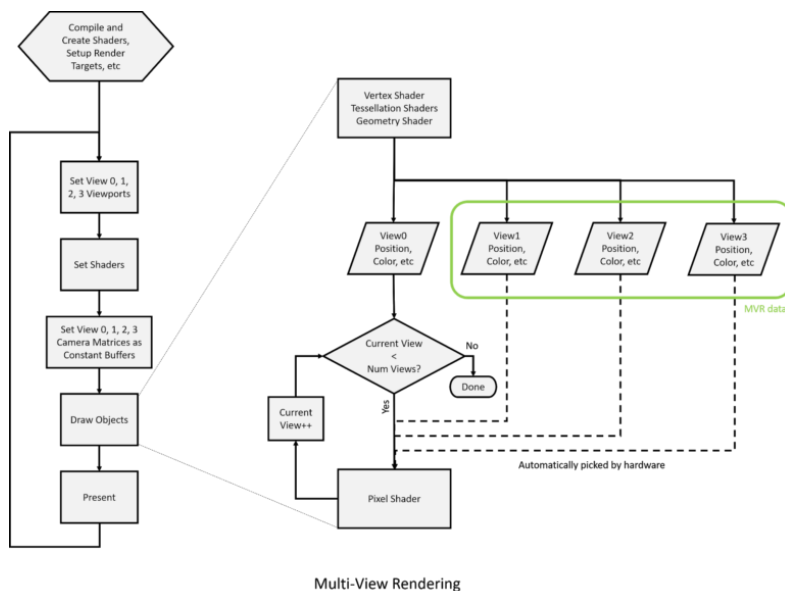
低延迟的多视图渲染管线

- 多视图渲染需要一次渲染**4个或更多视图**，这为**VR延迟**带来了挑战。通过同步多投影算法，一次性计算视图的通用属性，再为每个视图单独进行像素着色，从而避免了CPU与GPU的重复计算，提高渲染效率。



低延迟的多视图渲染管线

- NVIDIA Turing多GPU架构目前已支持多视图渲染特性，通过API，开发者可以在单个绘制调用中渲染多达 4 个视图，并且不显著增加CPU与GPU的计算负担。



NVIDIA Turing架构的多视图渲染流程图

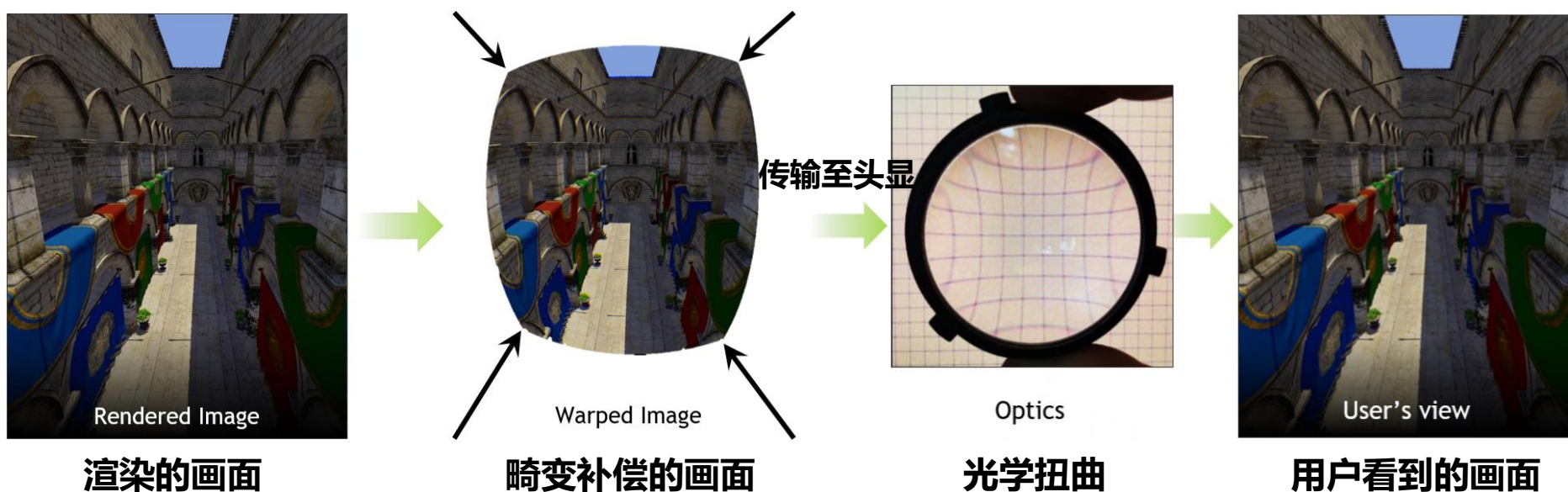
多视图渲染特性可在一次绘制中**渲染4个视图**，有力支持**大视野VR渲染**

针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- **多分辨率渲染**：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

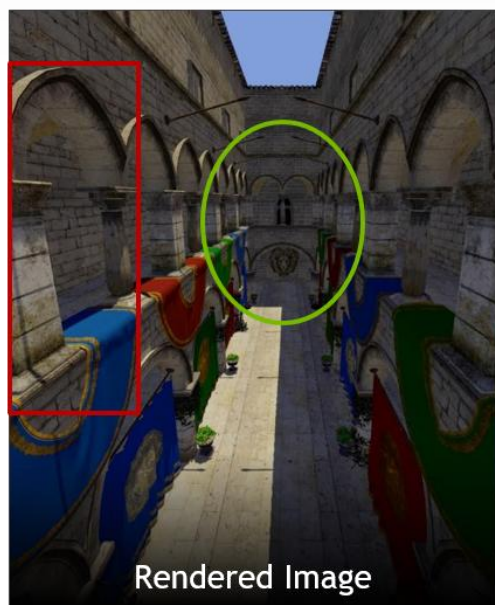
VR渲染中的畸变补偿

- 由于 VR 头显贴近眼球，需要一组光学镜片帮助用户对焦。但因为镜片会导致画面畸变，所以在传输至头显前，渲染引擎需要对画面加入“反向扭曲”来进行畸变补偿。



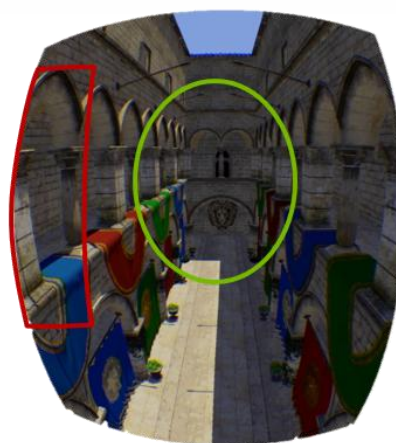
VR渲染中的畸变补偿

- 畸变补偿后的画面信息，比 GPU 渲染的画面少很多。**所以 GPU 本质做了很多无用功。**



Rendered Image

渲染的画面



Warped Image

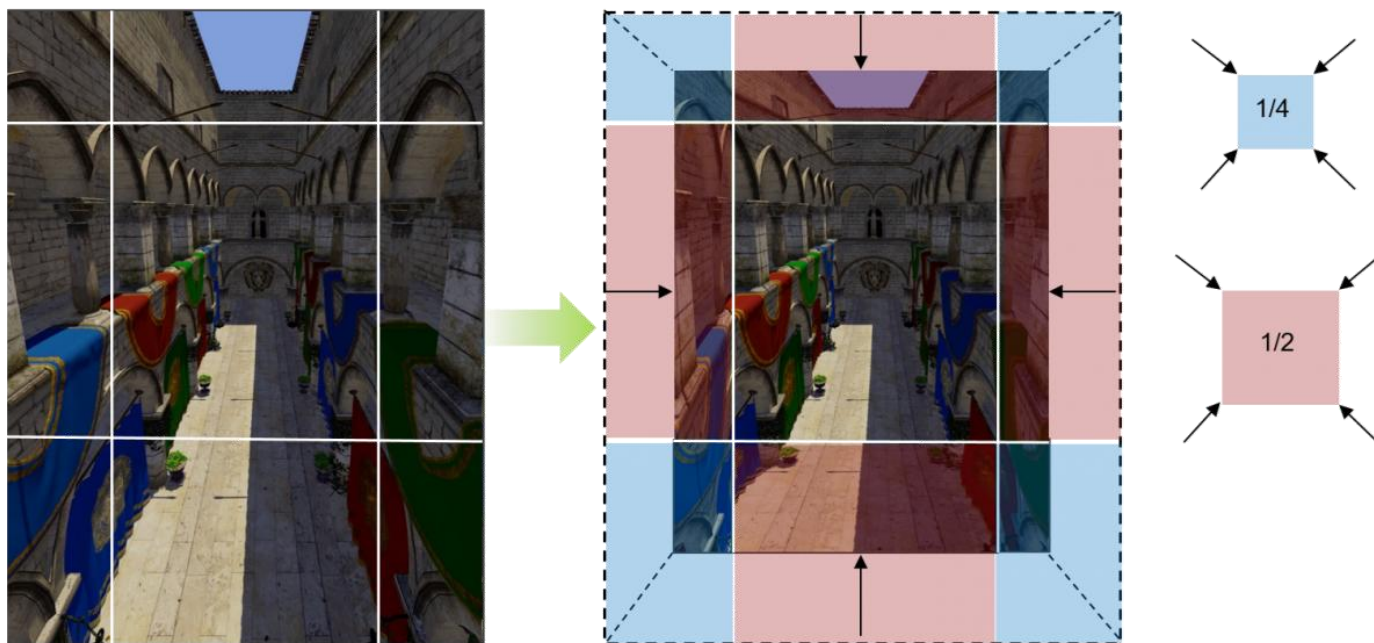
畸变补偿的画面

畸变补偿的画面中心保持不变，但边缘被压缩了很多，并且角落几乎完全消失了。

- 原始图像边缘的渲染是过度的，**许多像素根本不会出现在显示器上**，可以省略对它们的渲染以节省算力。

多分辨率渲染 (Multi-Res Shading)

- 将图像分成**多个视口**（如下图），保持中心视口大小相同，但缩小边缘周围的所有视口。这样一来：
 - 减少着色像素数量，渲染速度更快
 - 更好近似最终想要生成的扭曲图像，不浪费过多的像素

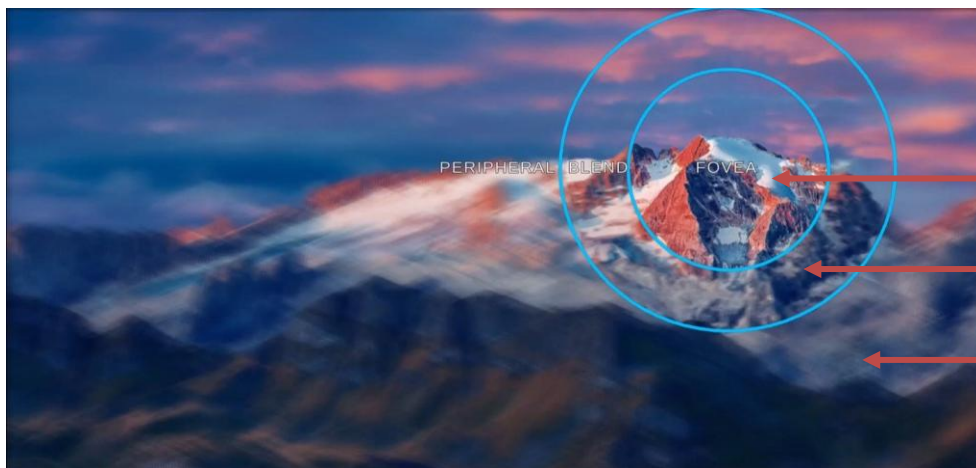


针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- **注视点渲染**：仅注视部分高分辨率，降计算负载
- AI超分辨率技术：降计算/传输负载，提分辨率

人眼的注视力视觉机制

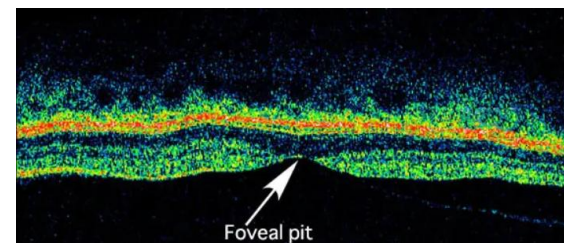
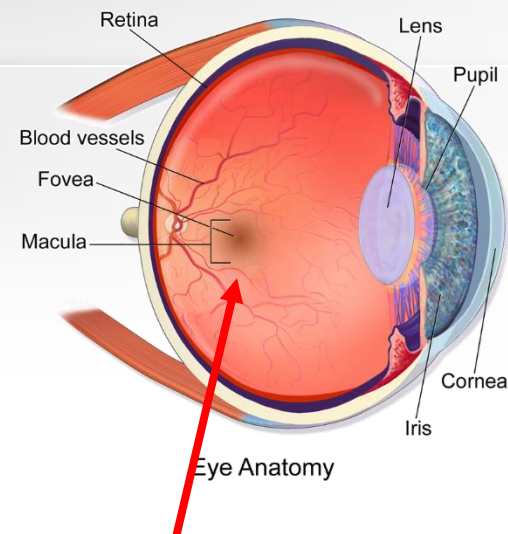
- 视网膜上负责高清成像的**视锥细胞并不均匀**，绝大多数的视锥细胞都分布在了视网膜中央一个叫作**中央凹**的区域。这意味着人眼看到的图像只有中间区域是清晰的，周围区域则为模糊或黑白的。**为周围区域渲染同等质量的高清图像是算力的浪费。**



中央凹区域成像

视锥与视杆细胞混合区域成像

仅视杆细胞(Rods)的外围区域成像



绝大多数视锥细胞都分布在中央凹这个很小的区域

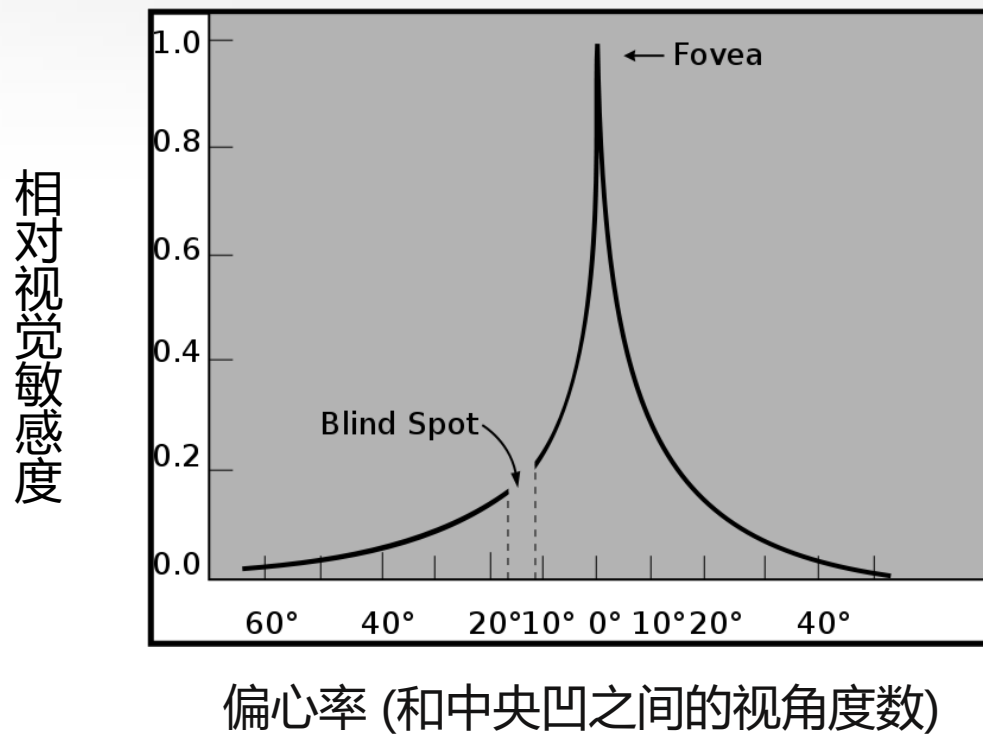
注视点渲染 (Foveated rendering)

- **注视点渲染**是一种提升渲染性能，减少GPU计算的技术。
- **仅高质量渲染用户正在注视的场景区域**，而对用户注视点以外的区域采取低质量渲染，从而节约GPU算力。



渲染质量随注视点移动而变化(动图)

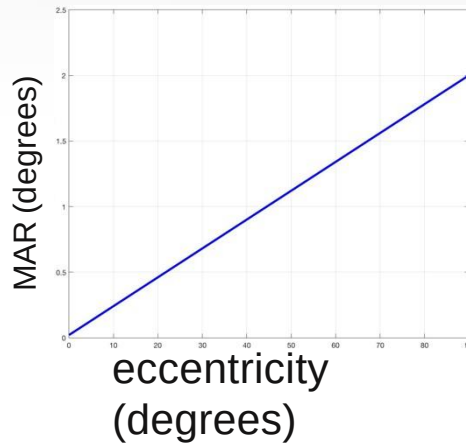
视网膜上的视觉敏感度



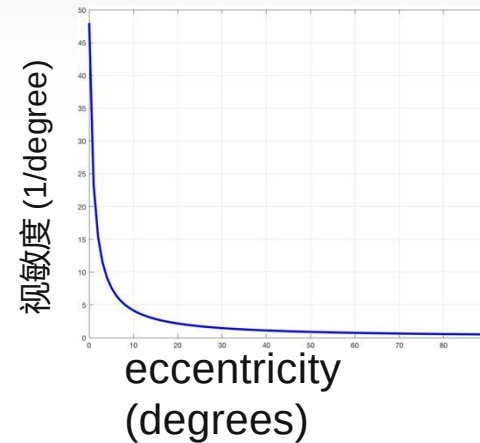
视网膜上不同位置的视觉敏感度不同，中心凹处最高，越靠近边缘越低。

视网膜上的视觉敏感度

最小可分辨角度 (MAR)



视敏度 (1/MAR)



最小可分辨角度
(MAR)

斜率



$$\omega = me + \omega_0$$



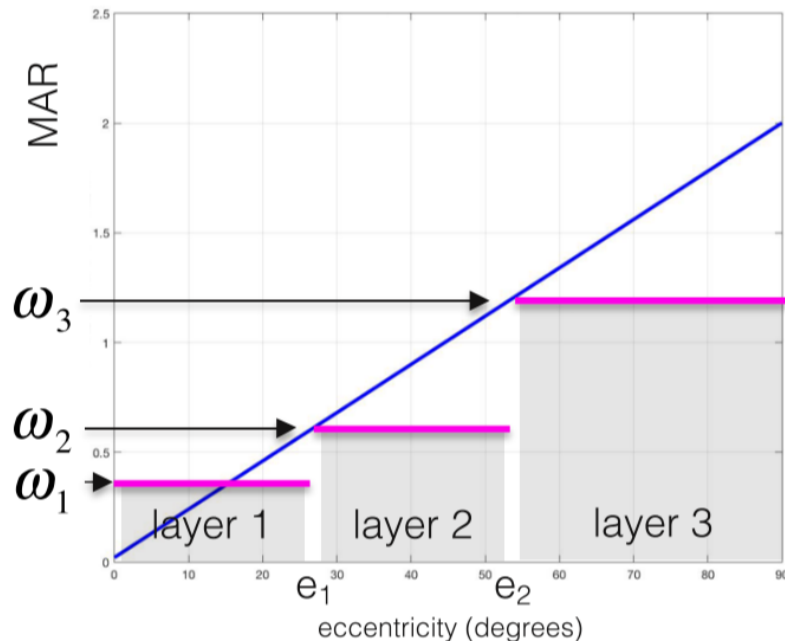
偏心率 (度) 中心凹处的最小可分辨角度

$$\omega_0 = (1/48)^\circ$$

$$m = 0.022 - 0.034$$

注视点渲染

将图像分为三层，如中心凹层（1）、中层（2）、外层（3）



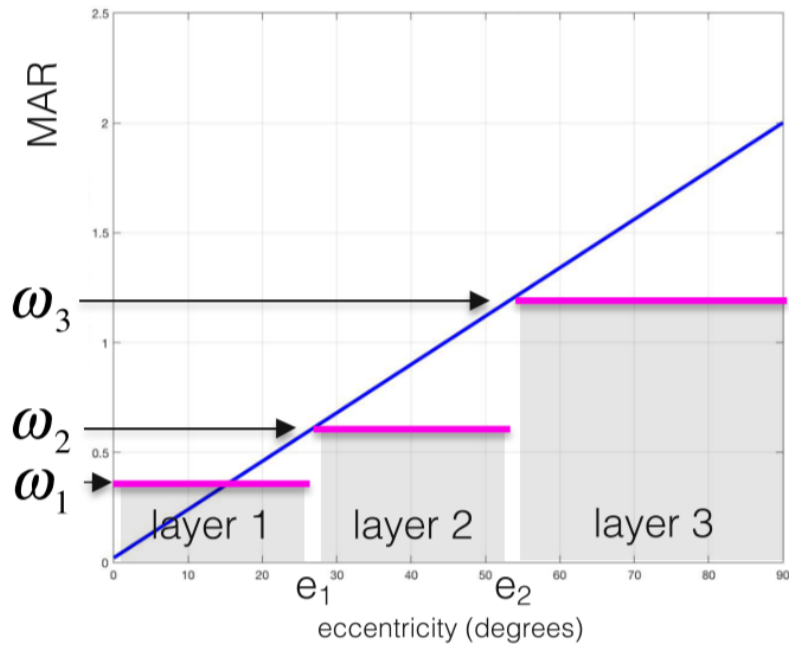
$$e_i = \frac{i}{n} \cdot \frac{fov}{2}$$



$$e_1 = \frac{fov}{6}$$

$$e_2 = \frac{fov}{3}$$

注视点渲染



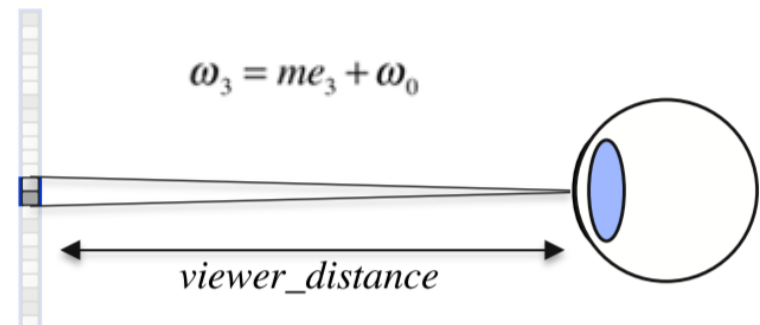
ω_1 是屏幕能够呈现的最小可视角度

$$\omega_1 = \frac{\text{degrees}}{2 \cdot \text{pixel_size}}$$

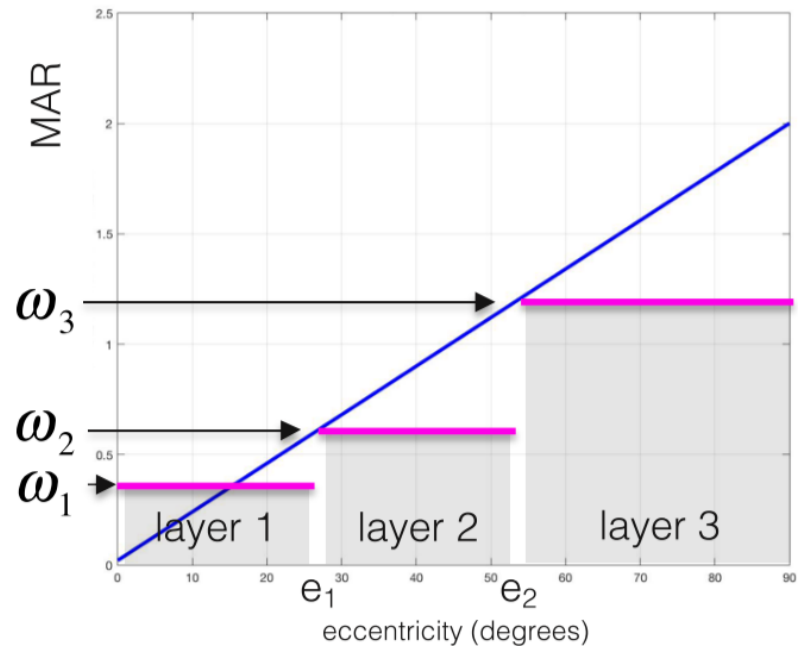
$$\omega_1 = 2 \tan^{-1} \left(\frac{\text{screen_size}}{\text{screen_resolution} \cdot \text{viewer_distance}} \right) \cdot \frac{360}{2\pi}$$

$$\omega_2 = me_2 + \omega_0$$

$$\omega_3 = me_3 + \omega_0$$



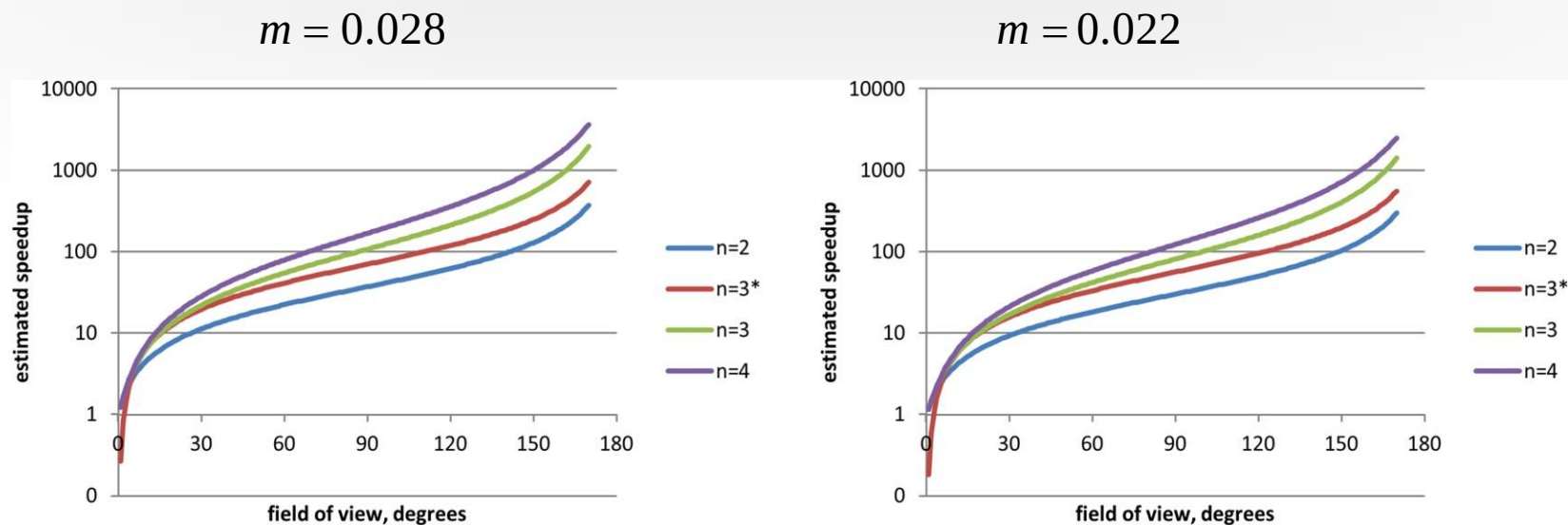
注视点渲染



将 ω_2 和 ω_3 转换为等效像素的大小。

$$blur_radius_in_px = viewer_distance \cdot \tan\left(\frac{\omega}{2} \cdot \frac{2\pi}{360}\right)$$

注视点渲染带来的性能增益



n : 划分的层数

speedup: 总显示像素数 / 需要渲染的像素数

结论: 对于大视野、高分辨率的显示情况, 我们可以渲染更少数量的像素!

注视点渲染的类型

- **静态注视点渲染**(fixed foveated rendering): 在不具备眼动追踪功能的VR头显上，固定地选择画面的中心作为高分辨率渲染区域，而画面四周作为低分辨率渲染区域。
- 如 Oculus Quest 2 就具备这种静态注视点渲染功能。



静态注视点渲染

注视点渲染的类型

- **动态注视点渲染**(dynamic foveated rendering): 在有眼动追踪功能的头显上, 根据用户视线方向动态选择注视点中心为高分辨率渲染区域, 而四周为低分辨率渲染区域。
- HTC Vive Pro Eye是较早支持动态注视点渲染的头显。



动态注视点渲染

注视点渲染

• 静态与动态注视点渲染的优劣比较

注视点渲染	优点	缺点
静态	不需要任何额外的硬件，但仍可提供一些渲染性能加速	如果用户将视线从显示器中心移开，图像质量就会明显下降
动态	可以根据眼睛某一时刻注视的位置来渲染区域，允许对屏幕内容进行更精细的渲染	需要额外的硬件来跟踪人眼的注视。如果眼球追踪失败，视觉体验就会被破坏，沉浸感也会消失



HTC Vive Pro Eye中集成的眼动追踪可以实现动态注视点渲染

动态注视点渲染的软硬件发展

- 作为极具潜力的VR渲染技术，动态注视点渲染有望节约**90%的GPU算力**[1]。但受到**眼动追踪准确度**等因素的影响，该技术目前仍在发展的初始阶段：
 - Tobii Spotlight目前可以使VR头显上的帧率提升78%，功耗降低10%，GPU负载平均节省 66% [2]。
 - NVIDIA 的变速率超级采样 (Variable Rate Supersampling, VRSS)原是一种提高图像中心质量的静态注视点渲染的技术，但近期推出的VRSS 2通过与Tobii合作使实现了应用程序的动态注视点渲染 [3]。

[1] <https://vr-expert.com/what-is-foveated-rendering-and-what-does-it-mean-for-vr/>

[2] <https://www.tobii.com/products/integration/xr-headsets/foveation-technology>

[3] <https://developer.nvidia.com/blog/nvidia-vrss-2-dynamic-foveated-rendering-no-assembly-required/>

动态注视点渲染的软硬件发展

- 开启NVIDIA VRSS 2实现应用程序动态注视点渲染：



NVIDIA VRSS 2通过动态注视点渲染实现8倍的着色效率。蓝色注视点区域由眼睛注视跟位置确定，具有最高的图像质量。（动图）

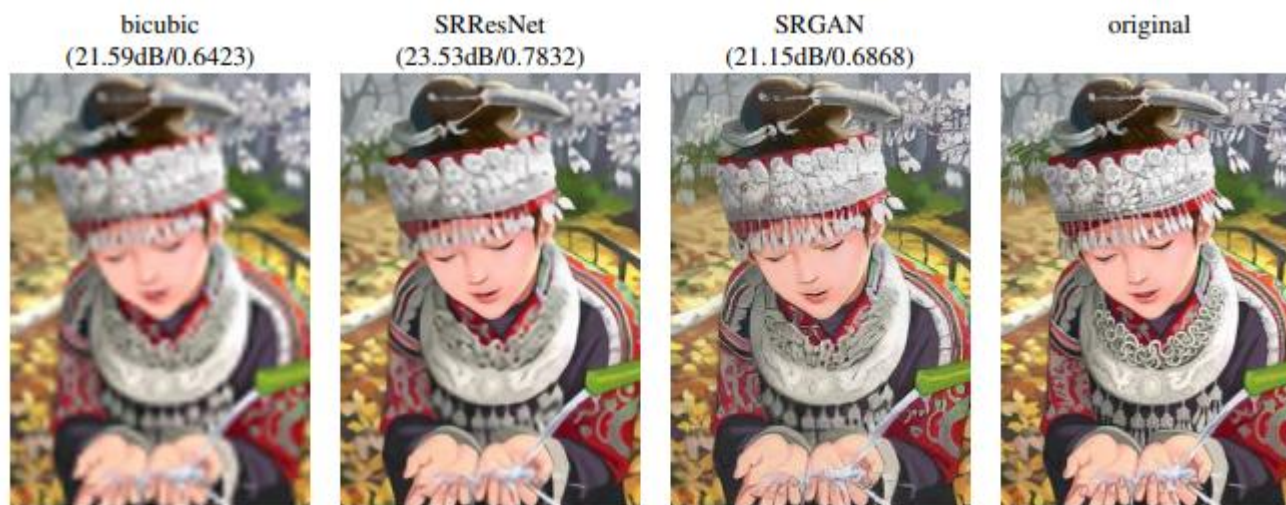
针对VR的视觉渲染增强

- 时间扭曲与异步时间扭曲：降延迟，提帧率
- 异步空间扭曲：利用运动合成帧，提帧率
- 多路复用渲染：渲染任务分配给多GPU，提帧率
- 重投影技术：减少CPU重复计算，降计算负载
- 多视图渲染：非共面显示器中的渲染，大视野
- 多分辨率渲染：减少边缘计算，降计算负载
- 注视点渲染：仅注视部分高分辨率，降计算负载
- **AI超分辨率技术**：降计算/传输负载，提分辨率

AI超分辨率技术

- **图像/视频超分辨率 (Super-Resolution)**

- 传统方法：插值、数据驱动学习低-高映射
- 现代深度学习主流架构：CNN[1]、GAN[2]、Transformer[3]、Diffusion[4]等



[1] Dong, Chao, et al. "Image super-resolution using deep convolutional networks." IEEE transactions on pattern analysis and machine intelligence 38.2 (2015): 295-307.

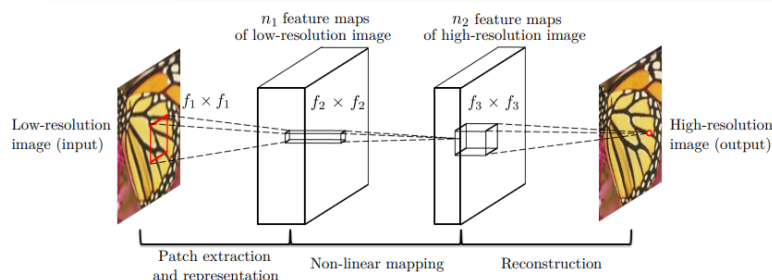
[2] Ledig, Christian, et al. "Photo-realistic single image super-resolution using a generative adversarial network." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017.

[3] Liang, Jingyun, et al. "Swinir: Image restoration using swin transformer." *Proceedings of the IEEE/CVF international conference on computer vision*. 2021.

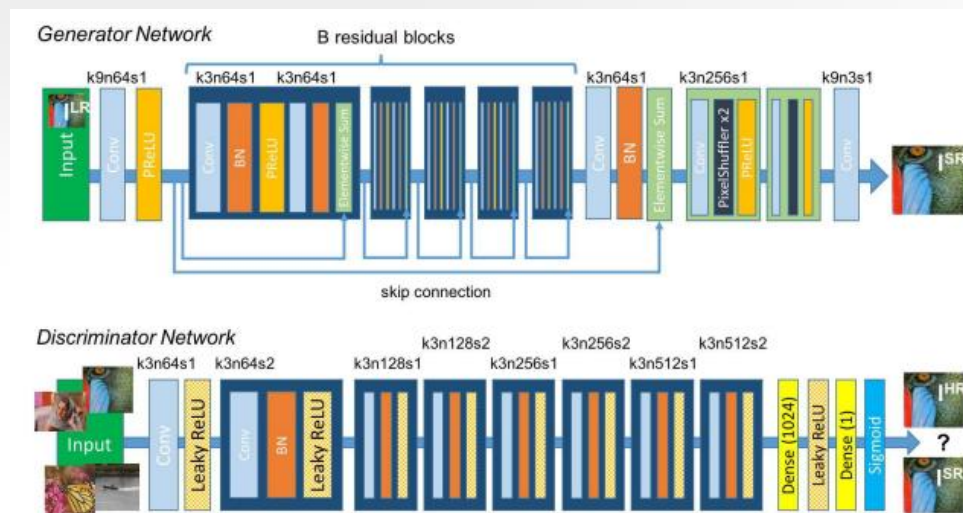
[4] Li, Haoying, et al. "Srdiff: Single image super-resolution with diffusion probabilistic models." *Neurocomputing* 479 (2022): 47-59.

AI超分辨率技术

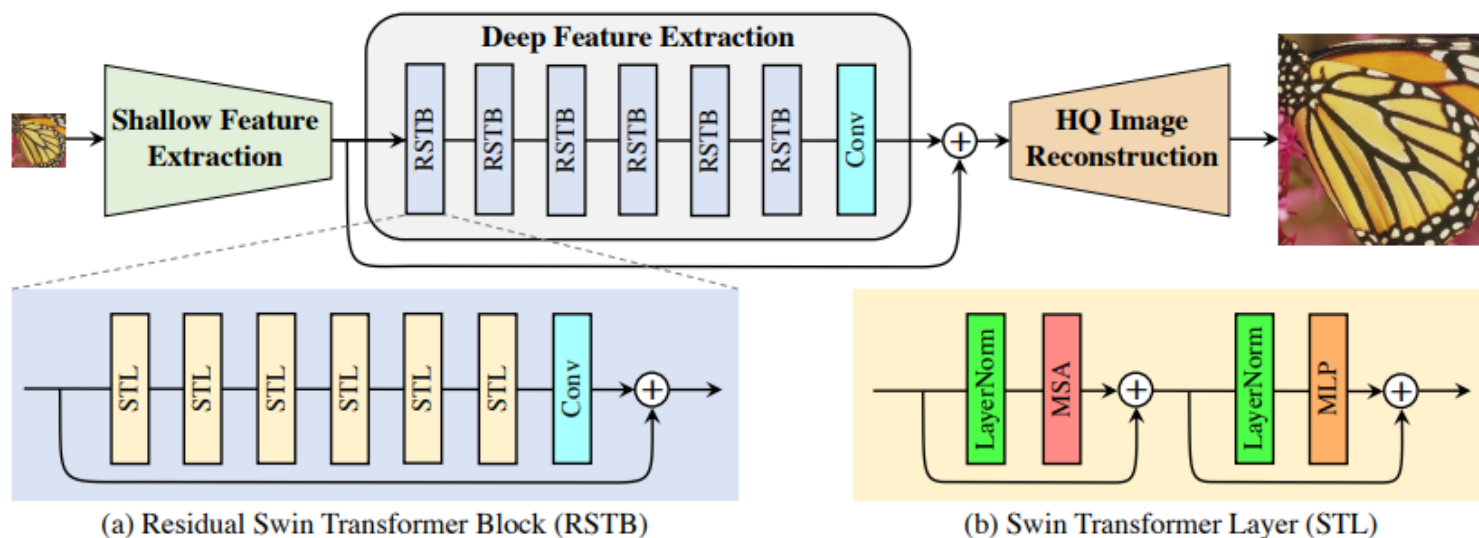
• 基于深度学习的超分



SRCNN (Dong et al. 2015)



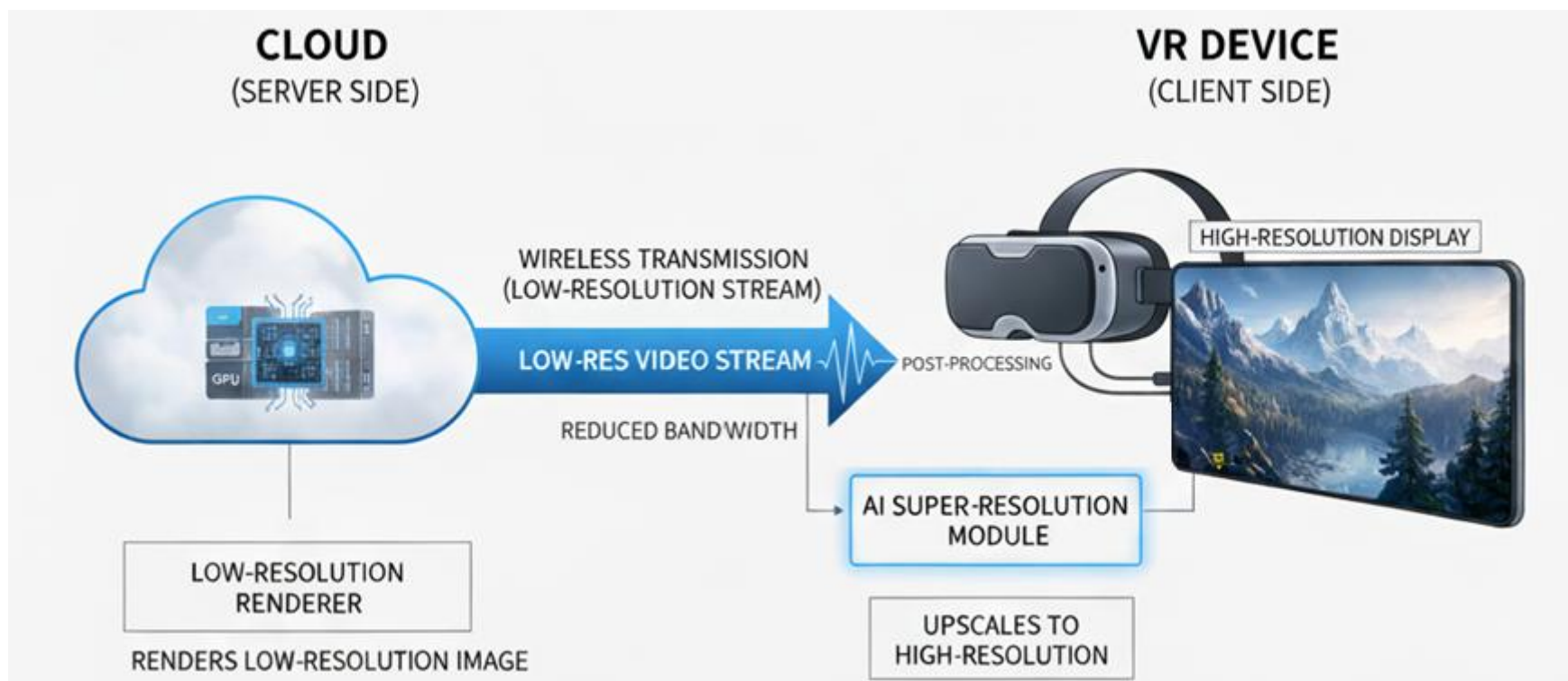
SRGAN (Ledig et al. 2017)



SwinIR
(Liang et al. 2021)

AI超分辨率技术

- **AI超分辨率在VR渲染中的应用**
 - 云侧渲染低分辨率图像/视频流，端侧超分
 - 降低云侧计算负载、降低传输负载和延迟



AI超分辨率技术-案例

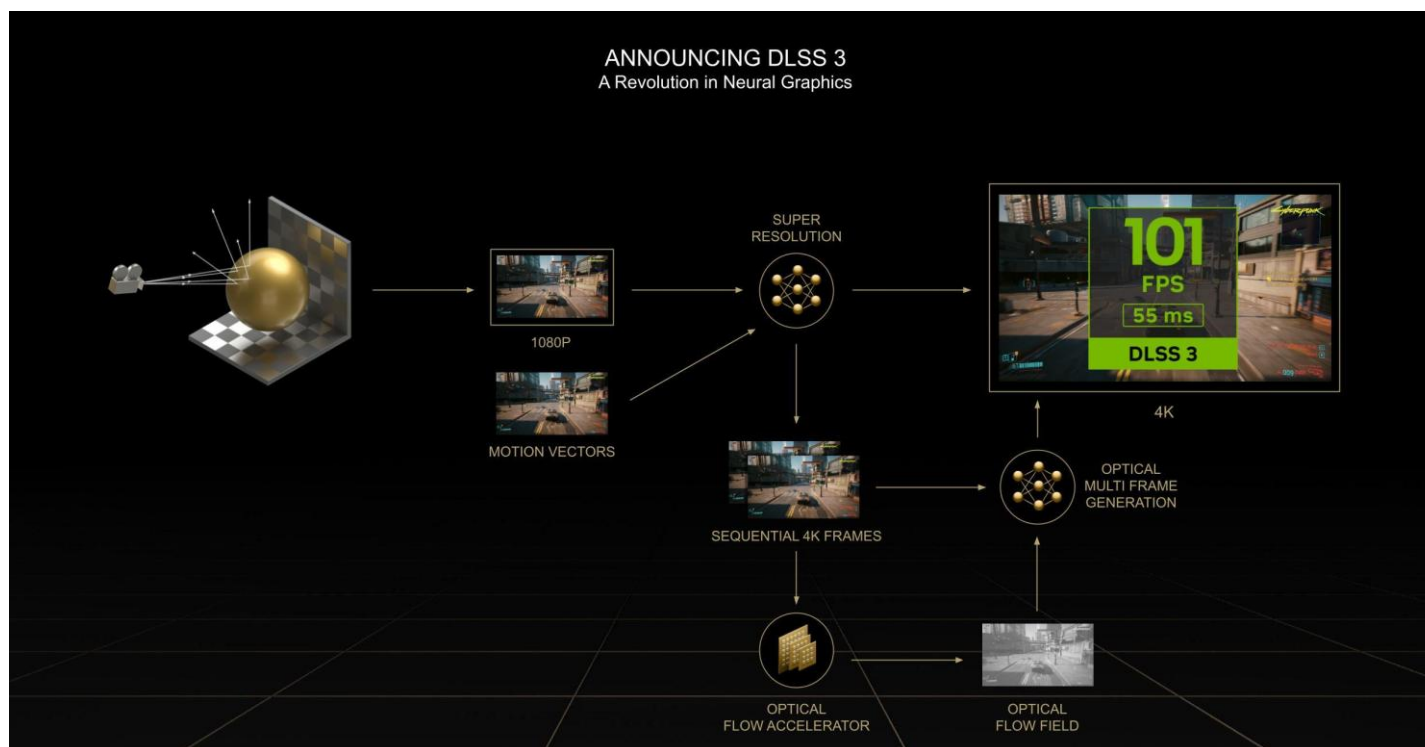
- **NVIDIA DLSS** (Deep Learning Super Sampling)
 - 4.0之前用CNN架构, 4.0使用Transformer架构
 - 除超分外, 还支持插帧、光线重建、抗锯齿等



AI超分辨率技术-案例

• NVIDIA DLSS

- 超分同时考虑视频流和历史运动向量
- 超分后经过光流生成再合成

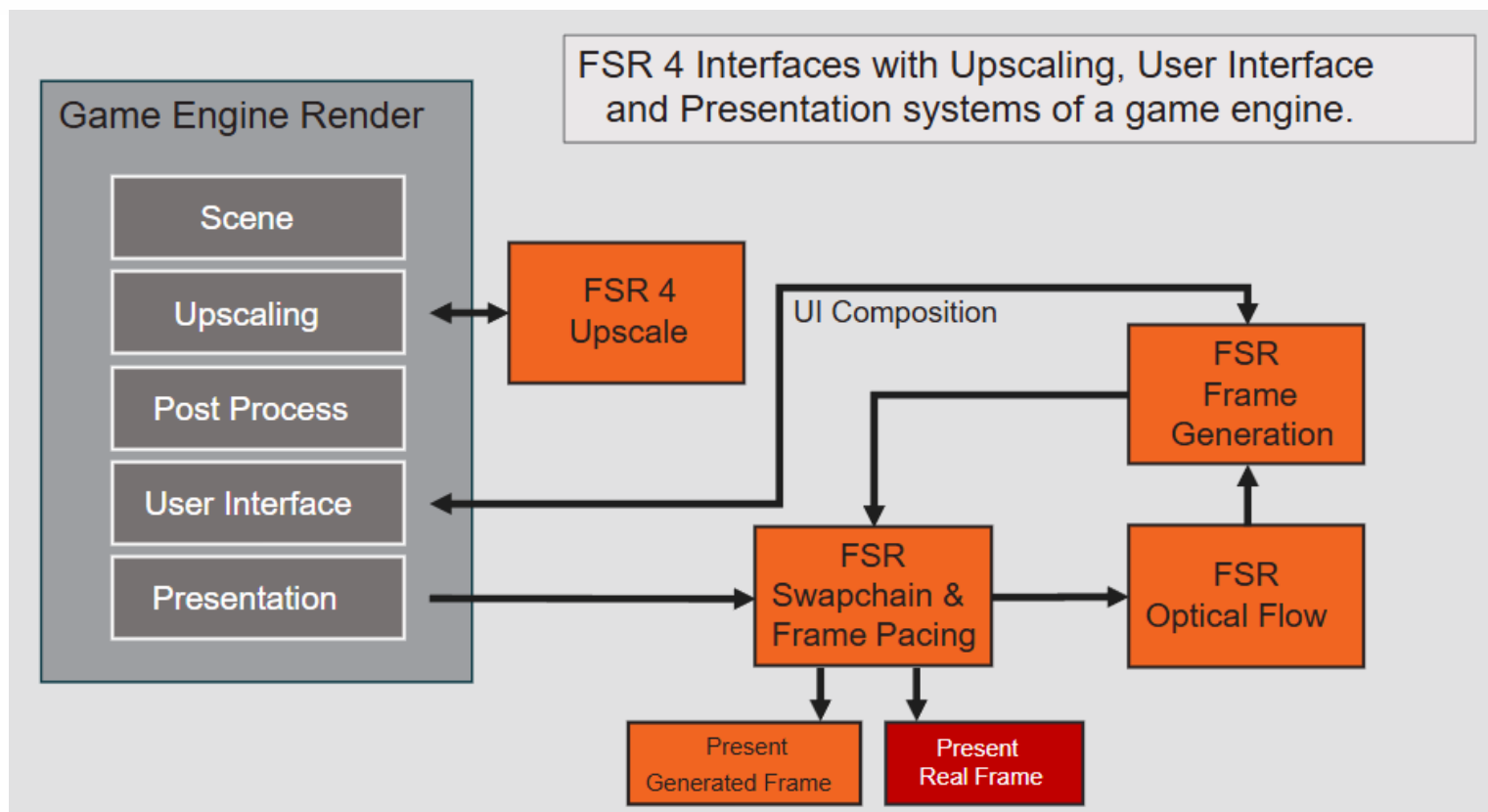


DLSS 3架构示意

<https://developer.nvidia.com/blog/accelerating-ultrarealistic-game-development-with-nvidia-dlss-3-and-rtx-path-tracing/>

AI超分辨率技术-案例

- **AMD FSR (FidelityFX Super Resolution)**
 - 4.0之前使用传统方法, 4.0首次使用AI超分



想一想

- 你还有什么思路来减少VR体验的渲染延迟？



目前主流的HTC VIVE VR系统套装

目录

01 VR渲染引言

02 视觉渲染基本方法

03 针对VR的视觉渲染增强

04 VR中的其他感官渲染

听觉渲染

- **基于真实采样生成音频**

- 如今的计算机不足以完全模拟真实物理发声过程，因此常见的做法是录制真实声音样本后加以处理再播放

- **常见后期处理方式**

- 滤波：使采样部分某些频率的振幅升高或降低
- 频谱组合：组合/删减不同声音的频谱来创造复合声音
- 粒度合成：使用细粒度的声音片段叠加合成更丰富的复合声音



听觉渲染

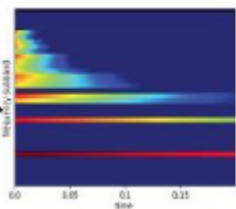
• 基于合成生成音频

- 频谱合成：观察采样声波的频谱（频率和相应幅度），然后模拟相似频谱来合成原始声音。
- 基于物理模型合成：基于产生声音的物体的物理特性来生成声音。比如通过建模发动机参数与声波的关系，就可以改变这些参数来渲染新的引擎声。



Impact Event

模态分析



Ground-truth



Our result

触力觉渲染

- **基于皮肤感知的渲染**

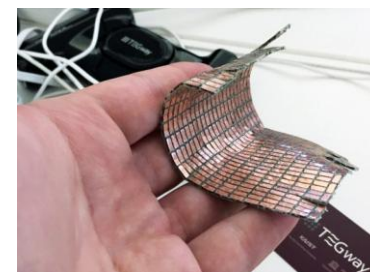
- 振动：偏心马达、线性谐振器等模拟
- 压力：针状阵列、气囊模拟物体形状和硬度
- 温度：热电阵列模拟灼热或冰冷



偏心振动马达

- **基于肌肉/关节的渲染**

- 力反馈：机械臂、外骨骼等设备提供阻力、推力，模拟物体的重量、惯性和碰撞等



热电阵列

前庭感觉渲染

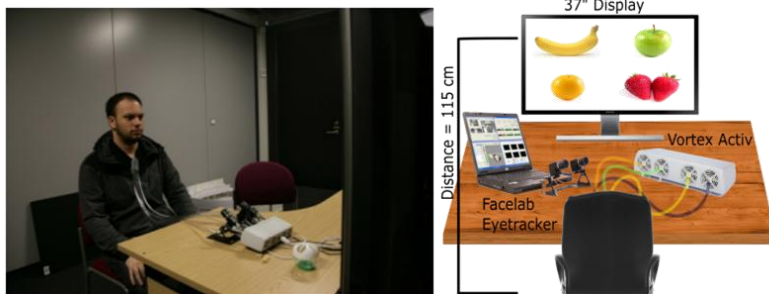
- 前庭是人体重要的平衡器官，可以感知方位和空间。**让用户前庭和身体的运动感知与VR中的体验一致**，可以降低眩晕感、提升沉浸体验。
- 比如在模拟飞行和模拟驾驶中，使用动态驾驶舱为用户前庭提供更多的运动信息，产生更逼真的运动体验。



使用动态驾驶舱进行前庭感觉渲染[1]

嗅觉渲染

- 在VR渲染中还处于起步阶段
- 气味主要由气体中的分子产生，捕捉气味通常需要对空气进行采样，梳理出不同的成分，然后在实验室中复制
- VR中的嗅觉渲染主要是通过控制**释放气味分子来模拟气味**



Harvey, C., Bashford-Rogers, T., Debattista, K., Doukakis, E., & Chalmers, A. (2018, February). Olfaction and selective rendering. In Computer Graphics Forum (Vol. 37, No. 1, pp. 350-362).

OVR公司推出的气味架构可模拟约250种气味



THANKS!

清华大学计算机系

张松海

shz@Tsinghua.edu.cn